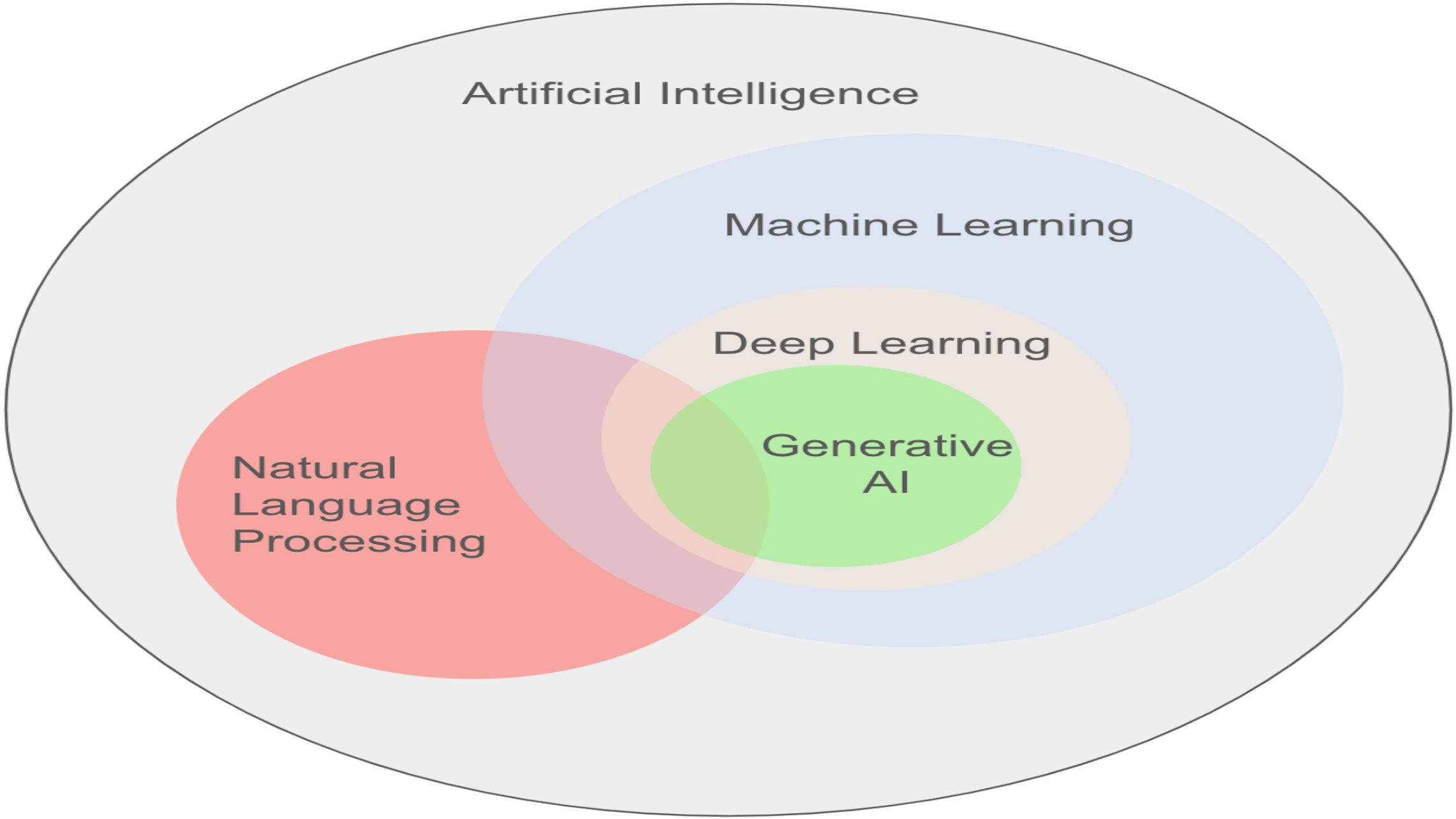
Prompt Engineering for GenAI

23CI72

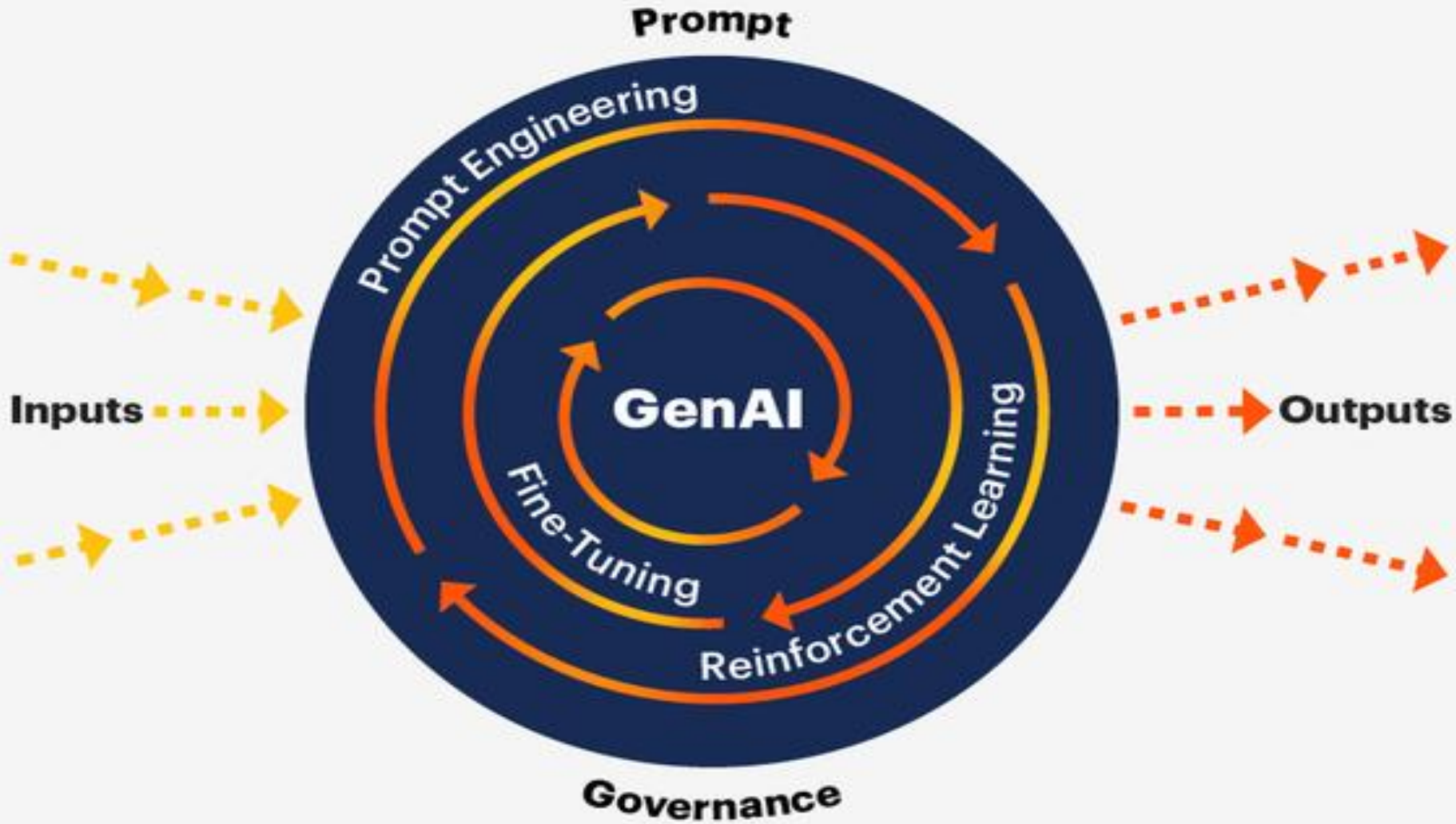
Dr. A N Ramya Shree
Dept. of CSE(AI&ML)

What is Generative AI?

- **Generative Artificial Intelligence (Generative AI)** is a branch of artificial intelligence that focuses on creating **new content** such as text, images, audio, video, computer programs, designs, and synthetic data by learning patterns from existing data.
- Traditional AI is generally designed to **analyze, classify, predict, or make decisions**, whereas Generative AI can **produce new outputs** that resemble the data on which it was trained.


Artificial Intelligence
Machine Learning
Deep Learning
Generative AI
Natural Language Processing

GenAI Framework



Source: Gartner
© 2024 Gartner, Inc. and/or its affiliates. All rights reserved. 2948768

Example

- A traditional AI model may determine whether an email is **spam or not spam**.
- A Generative AI model can **write a new email** based on a user's instructions.
- A traditional computer vision model may classify an image as a **cat**.
- A Generative AI model can **create a new image of a cat** that did not previously exist

Case Study

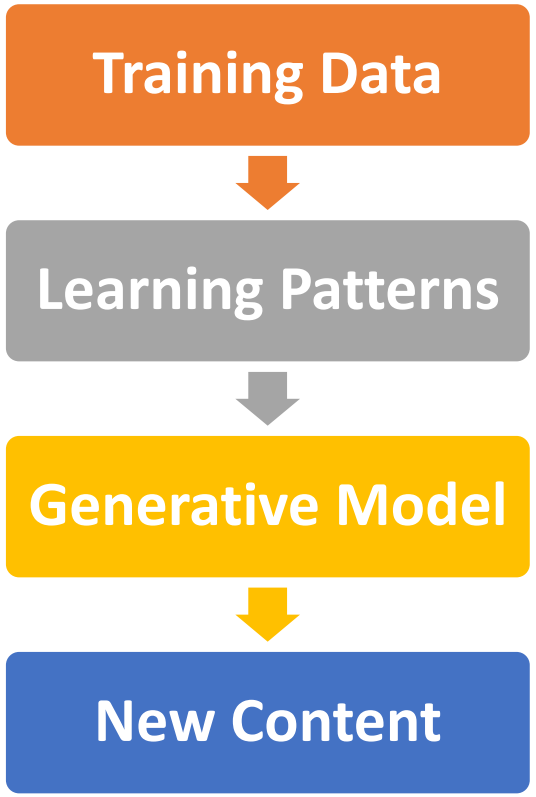
Generative AI learns the underlying patterns and relationships in a dataset.

Suppose a model is trained using thousands of images of handwritten digits. It learns characteristics such as:

- shapes
- curves
- strokes
- positions
- relationships between pixels

After learning these patterns, the model can generate a **new handwritten digit**.

Generative AI Representation



- A discriminative model approximately learns:
- P(y|x) where x is the input and y is the class or target.
- A generative model attempts to learn the underlying data distribution, represented as:
- P(x) or, in conditional generation P(x|y)

Generative vs. Discriminative AI

Aspect	Discriminative AI	Generative AI
Main purpose	Classification/prediction	Content generation
Learns	Decision boundary or relationship	Data distribution/patterns
Output	Label/value	New content
Example	Spam detection	Email generation
Image application	Cat vs. dog classification	Creating a new cat image
Common models	Logistic Regression, SVM, CNN	GAN, VAE, Transformer, Diffusion

Traditional AI to Generative AI

The transition from traditional AI to Generative AI represents a shift from **"understanding and predicting existing information"** toward **"creating new information."**

Traditional AI

Traditional AI generally receives an input and produces a decision, classification, prediction, or recommendation.

For example:

X→ AI Model → Y

Consider a disease-detection system:

$$\text{Medical Image} \rightarrow \text{CNN} \rightarrow \begin{cases} \text{Disease} \\ \text{No Disease} \end{cases}$$

The model does not normally create a new medical image.

It predicts a class.

Generative AI

Generative AI learns patterns in the training data and uses

those patterns to generate new content.

X→ Generative Model →New Content

For example:

- Prompt→ LLM→ Generated Text
- Text Prompt → Generative Model → Generated Image

Text Prompt

↓

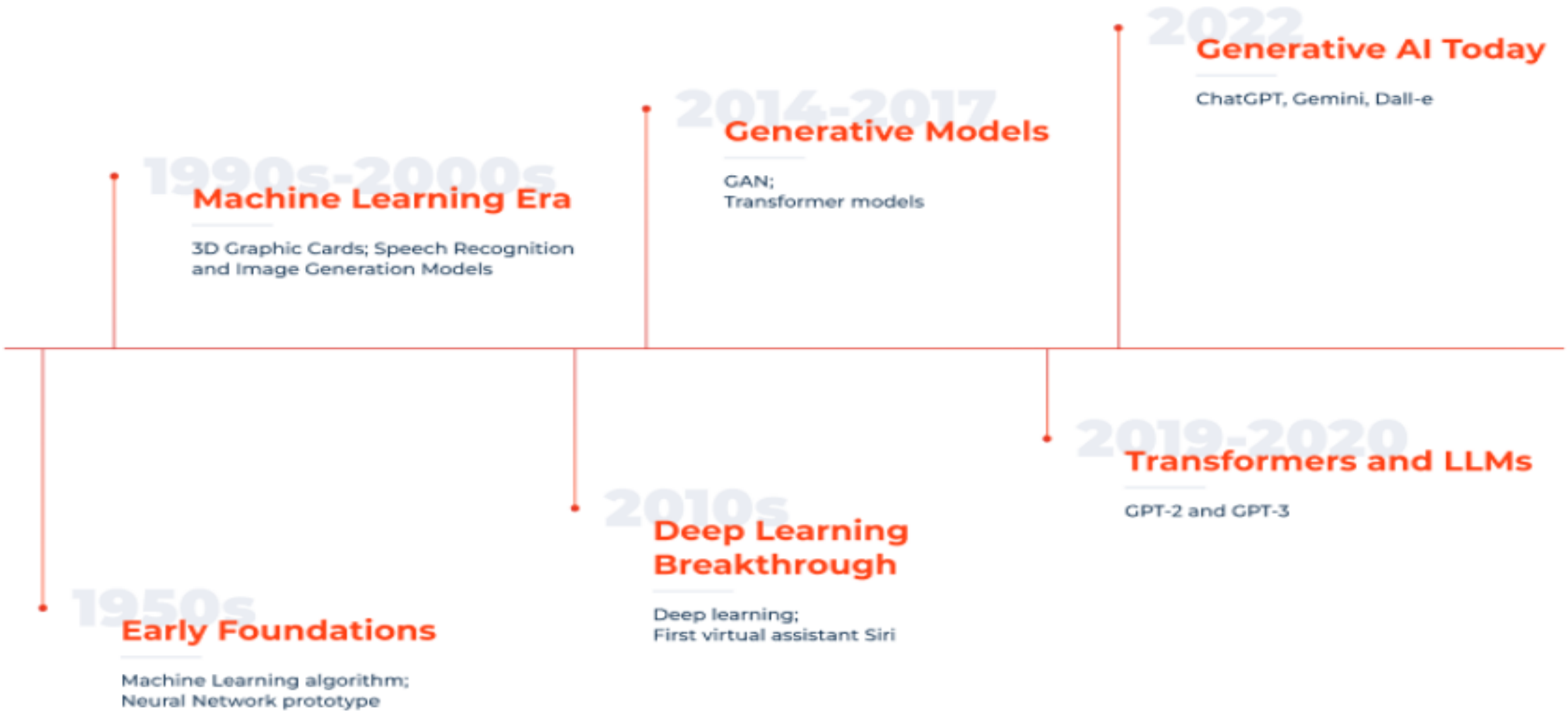
Generative Model

↓

New Cat Image

The generated image may not have existed in the training dataset

History of Generative AI



Generative Deep Learning

Deep learning was used not only to recognize data but also to **generate new data.** Important generative architectures included:

Autoencoders: Autoencoders learn compressed representations of data.

x→Encoder→z→Decoder→x^

where:

- **x = original input**
- **z = latent representation**
- **x^ = reconstructed output**
- **Variational Autoencoders (VAEs) extended this idea to probabilistic generation.**

Autoencoders

An **autoencoder** is a type of **neural network** that is trained to **encode** input data into a **compressed representation (encoding)** and then **decode** it back to a reconstruction that resembles the original input.

Structure of Autoencoders

1**. Encoder:** Compresses the input into a lower-dimensional space (latent space)
2. **Bottleneck/Latent Space:** The compressed, encoded representation (most informative features).
3 **Decoder:** Reconstructs the original input from the latent space.

The network is trained to minimize reconstruction error.

Input (x) → Encoder → Latent Space (z) → Decoder → Reconstructed Input ($\hat{x}$)



Figure 8-4. The autoencoder architecture attempts to construct a high-dimensional input into a low-dimensional embedding and then uses that low-dimensional embedding to reconstruct the input

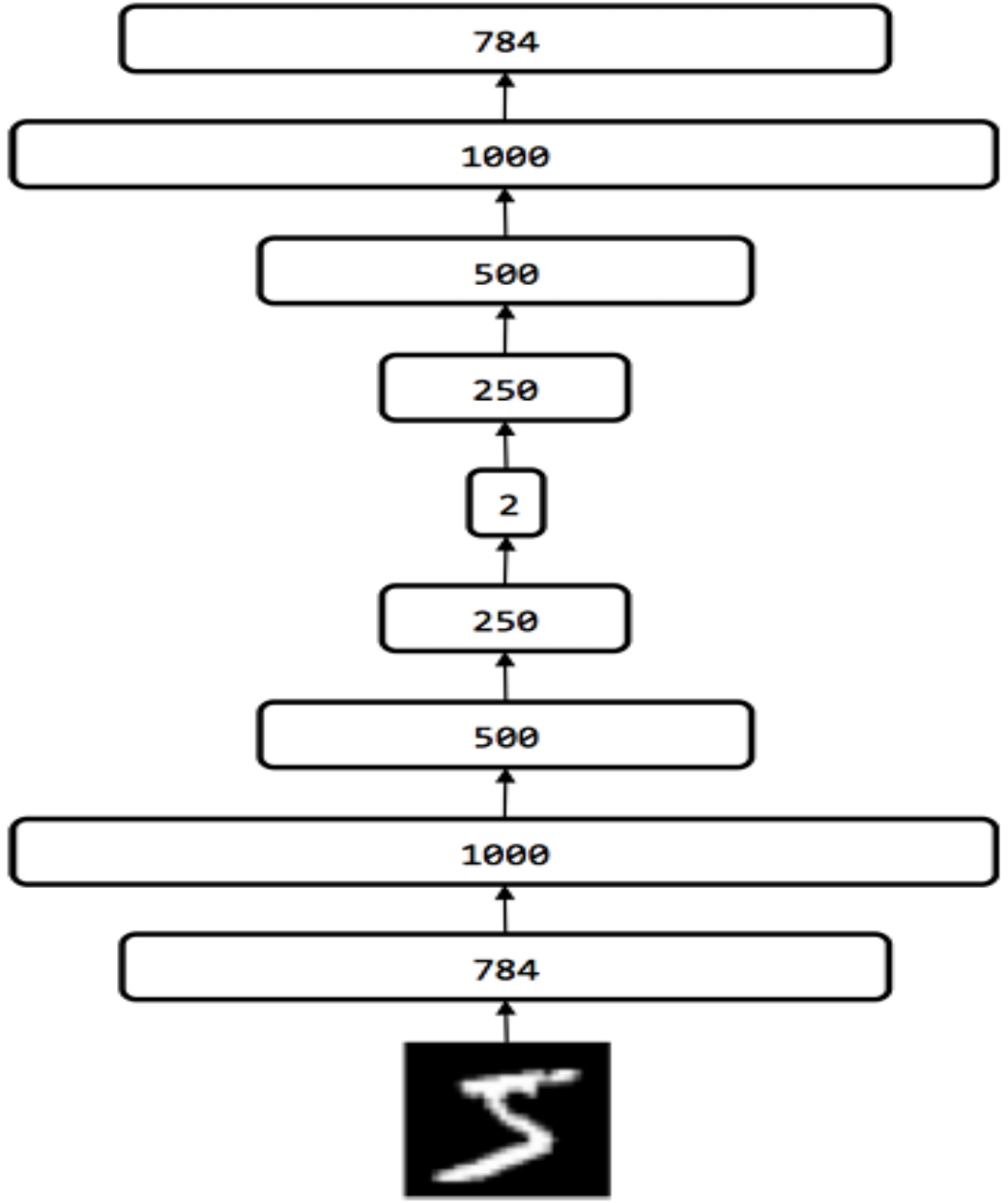


Figure 8-5. The experimental setup for dimensionality reduction of the MNIST dataset

Why Autoencoders?

- **Dimensionality Reduction:** They can learn a compact representation of data (similar to PCA but more flexible).
- **Denoising:** Can remove noise from data (denoising autoencoders).
- **Anomaly Detection:** If trained on normal data, high reconstruction error for anomalies.
- **Pretraining**: Used to pretrain deep networks before fine-tuning.

Representation Learning

- Noisy or inconsistent data hurts ML models with shallow understanding
- Solution: let the machine learn an abstract representation itself
- Turns high-dimensional data (e.g., text) into low-dimensional form

Benefits

- Easier pattern & anomaly detection
- Better grasp of data behavior
- Filters noise – helps supervised learning

Two things to remove

- Variance: output sensitivity; models must be resilient
- Entanglement: embeddings correlated with each other, hard to decipher

Information Bottleneck & Latent Variables

Information Bottleneck

- Compress the information flowing through the network
- Reduces dimensionality and complexity
- Irrelevant noise dropped; only features key to general concepts remain

Latent Variables (z)

- Hidden variables that shape the data distribution
- Much lower dimension than the input → compressed form
- Enable a generative model

Generate: $z \sim p(z) \rightarrow x \sim p(x \mid z)$

Sample a latent z, then draw a new observation x from the conditional distribution

Autoencoder Architecture

Encoder

Stacked layers, fewer neurons each; compresses input

Latent Space

Minimal "bottleneck" holding the compressed code

Decoder

Decompresses and reconstructs; compared with true values

Loss = Reconstruction loss $L(x, \hat{x})$ + Regularizer

Sensitive enough to reconstruct accurately, yet not so sensitive that it memorizes or overfits. A scaling parameter tunes the trade-off.

Types of Autoencoders

Incomplete

- Limits the number of hidden nodes to restrict information flow
- Penalized by reconstruction error
- Nonlinear generalization of PCA – learns a curved surface, not just a hyperplane

Sparse

- Limits active neurons via a sparsity penalty in the loss
- Different inputs activate different nodes
- Curbs memorization, keeps feature extraction
- Penalties: L1 regularization or KL divergence

Types of Autoencoders

Denoising

- Input corrupted with Gaussian noise; clean data is the target
- Learns a vector field pulling inputs to a low-dimensional region
- Works well near training data; error can be large far from it

Variational (VAE)

- Generative; latent space is continuous → easy sampling & interpolation
- Encoder outputs mean μ and std-dev σ; the code is sampled from that Gaussian
- KL loss pushes $\mu \to 0$, $\sigma \to 1$, spreading codes evenly around the origin

VAE: KL Divergence Loss

$$\text{KL loss} = \Sigma\ (\ \sigma i^2 + \mu i^2 - \log(\sigma i) - 1\ ),\ \ i = 1 \ldots n$$

$Xi \sim N(\mu i, \sigma i^2)$ vs. standard normal $N(0, 1)$

- KL divergence measures how different two probability distributions are
- Sum of KL divergences between each Xi and the standard normal
- Minimized when $\mu i = 0$ and $\sigma i = 1$
- Pushes encodings evenly around the latent-space center; penalizes clustering far from the origin

$$z \sim p(z)$$
$$x \sim p(x \mid z)$$

Generative process

- z: hidden latent variable, far lower-dimensional than x
- p(z): distribution we sample z from
- p(x | z): conditional distribution of data given z
- Sample z first, then draw a new x

$$\text{Loss} = L(x, \hat{x}) + \text{regularizer}$$

Autoencoder loss

- L(x, x̂): reconstruction loss – input x vs. output x̂; keeps the model responsive to inputs
- Regularizer: discourages memorization / overfitting
- A scaling parameter tunes the trade-off

$$X_i \sim N(\mu_i, \sigma_i^2)$$

VAE encoding

- Encoder outputs vectors μ (means) and σ (std. deviations), each of size n
- μ_i centers the encoding; σ_i sets how far it may deviate
- Code is sampled from this Gaussian, then passed to the decoder

$$KL = \sum_{i=1 \ldots n} \left(\sigma_i^2 + \mu_i^2 - \log \sigma_i - 1 \right)$$

KL divergence loss

- Measures distance of each $N(\mu_i, \sigma_i^2)$ from standard normal N(0, 1)
- Minimum at $\mu_i = 0$, $\sigma_i = 1$
- Spreads encodings evenly around the latent-space center

Types of Autoencoders

- **Vanilla Autoencoder:** Simple encoder-decoder structure.
- **Denoising Autoencoder:** Trained to remove noise.
- **Sparse Autoencoder:** Enforces sparsity in hidden units.
- **Variational Autoencoder (VAE):** Probabilistic framework, generates new data.

- Input: Each image (28x28) is flattened into a vector of size 784.
- Encoder: Compresses from 784 → 3 dimensions.
- Decoder: Reconstructs back to 784 dimensions.
- Loss: Mean Squared Error between original and reconstructed input.
- Training: Uses Adam optimizer.

A simple fully connected (dense) autoencoder

- Encoder:Linear (784 → 256) + ReLULinear (256 → 64) + ReLU
- Decoder:Linear (64 → 256) + ReLULinear (256 → 784) + Sigmoid (for pixel normalization to [0,1])
- Input: Flattened 28×28=784 vectors
- Output: Reconstructed flattened images

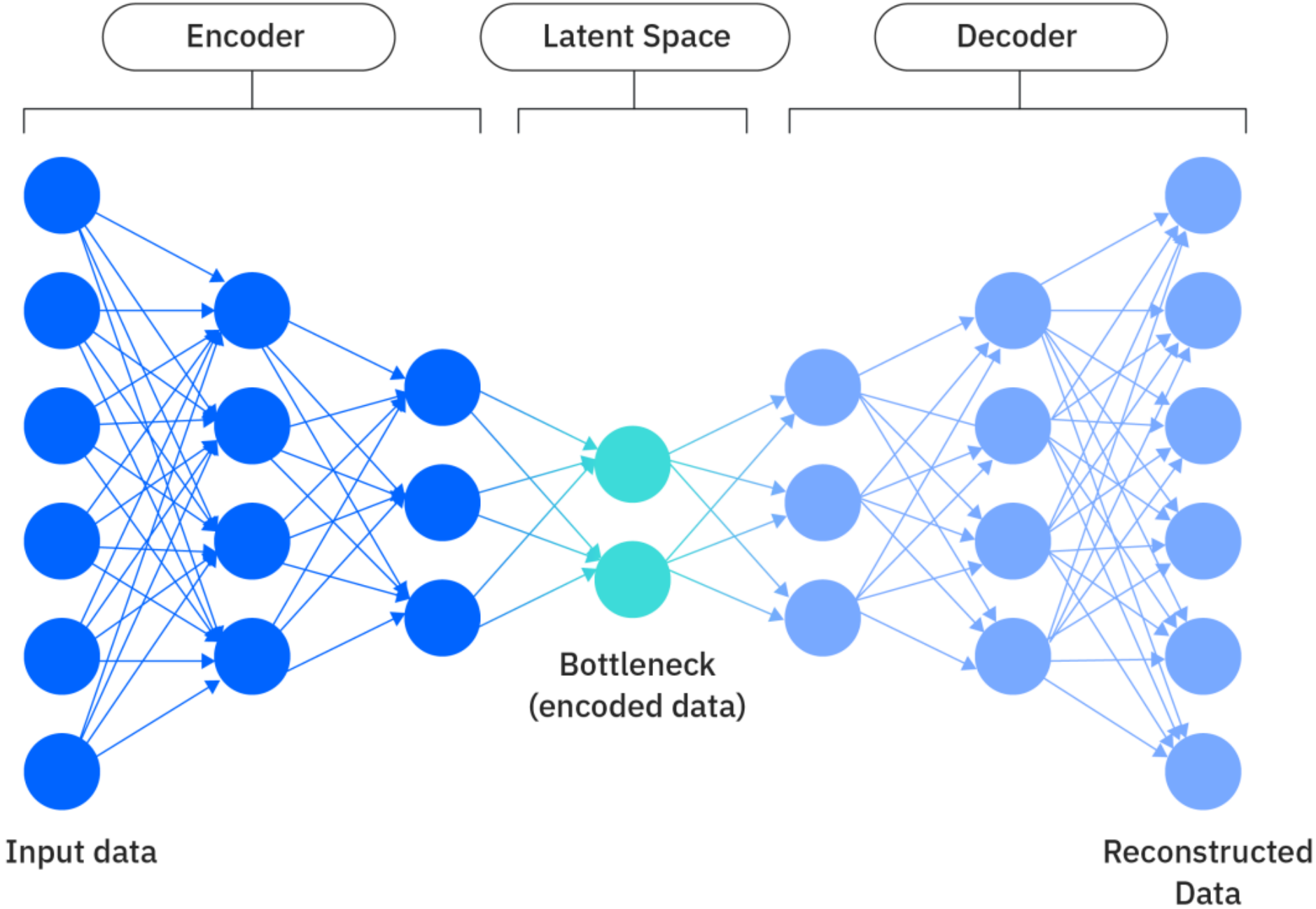
Undercomplete Autoencoder

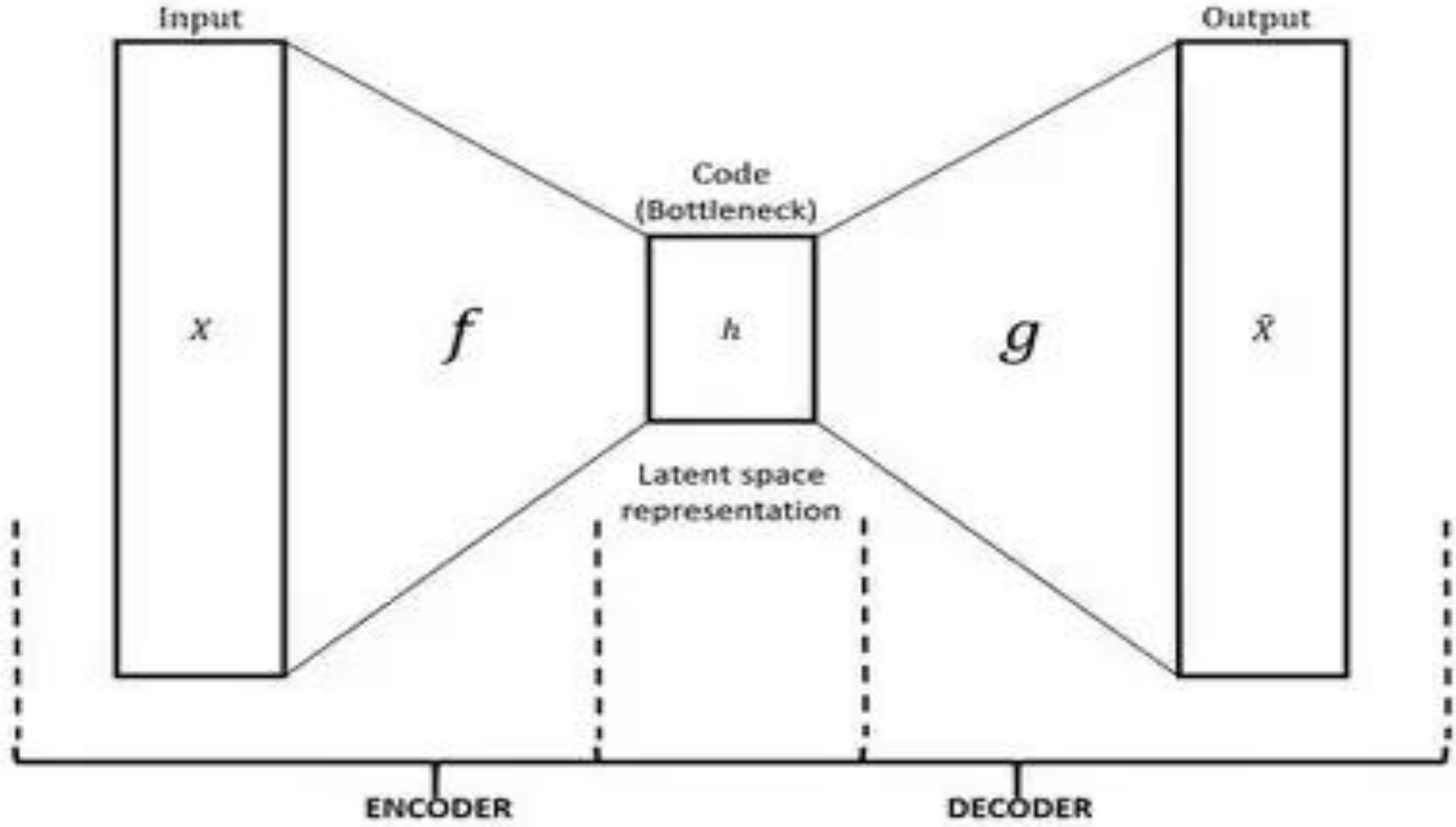
- An **undercomplete autoencoder** is designed with a **bottleneck (latent) layer** that has **fewer neurons** than the input layer.
- This forces the model to learn **compressed and essential features** of the input data.
- Input → Encoder → Latent Space (small) → Decoder → Output

Goal: Minimize reconstruction error

- X = Original input
- X^ = Reconstructed output

$$\mathcal{L} = \|X - \hat{X}\|^2$$


Encoder
Latent Space
Decoder
Bottleneck
(encoded data)
Input data
Reconstructed
Data


Input
Output
Code
(Bottleneck)
x
f
h
g
x̂
Latent space
representation
ENCODER
DECODER

- The **bottleneck** in an **undercomplete autoencoder** is the **central hidden layer** in the neural network that has a **smaller number of neurons** than the input and output layers.
- The **bottleneck** is the **latent space representation** that forces the model to **compress the input data**.
- This compression makes the network learn only the **most important features** needed to reconstruct the input, acting like an information filter.

- Forces the network to learn useful low-dimensional features.
- Acts as a dimensionality reduction technique, similar to PCA.
- Common in tasks like denoising, compression, or anomaly detection.
- Simple and effective for feature learning.
- Prevents overfitting by architectural design.
- Might not capture all important structures if the bottleneck is too small.

Structure of an Undercomplete Autoencoder

- Input Layer → 100 features
- Hidden Layers → 64 → 32 → **Bottleneck (10 neurons)**
- Decoder Layers → 32 → 64
- Output Layer → 100 features (same as input)

- If your input is a 28x28 image (784 features), and the bottleneck layer is of size 32, the model learns to represent each image with just 32 numbers that capture the most relevant information.
- Encoder: 784 → 128 → 64 → 32 ← Bottleneck
- Decoder: 32 → 64 → 128 → 784

Layer Size

- The layer size and depth of an autoencoder's architecture are critical for how well it learns useful representations and reconstructs input data.
- They directly influence the capacity, compression, and generalization ability of the model.
- **Layer size** refers to the number of neurons (units) in each layer of the autoencoder.

Input Layer: Matches the number of features in the data.

Hidden Layers: Intermediate layers in encoder and decoder.

Latent Layer (Bottleneck): Central compressed representation (latent code).

The size usually **decreases** toward the bottleneck and then **increases** symmetrically in the decoder.

Layer	Size (for 784-dim input)
Input	784
Hidden (1)	256
Hidden (2)	64
Latent Code	16 or 32
Decoder (1)	64
Decoder (2)	256
Output	784

Depth

Depth refers to the number of layers in the network.

Shallow Autoencoder:

One hidden layer in encoder and One in decoder.

Fast to train but may miss complex patterns.

Deep Autoencoder:

- Multiple hidden layers in encoder and decoder.
- Can learn hierarchical and more abstract features.
- Needs more data and training time.

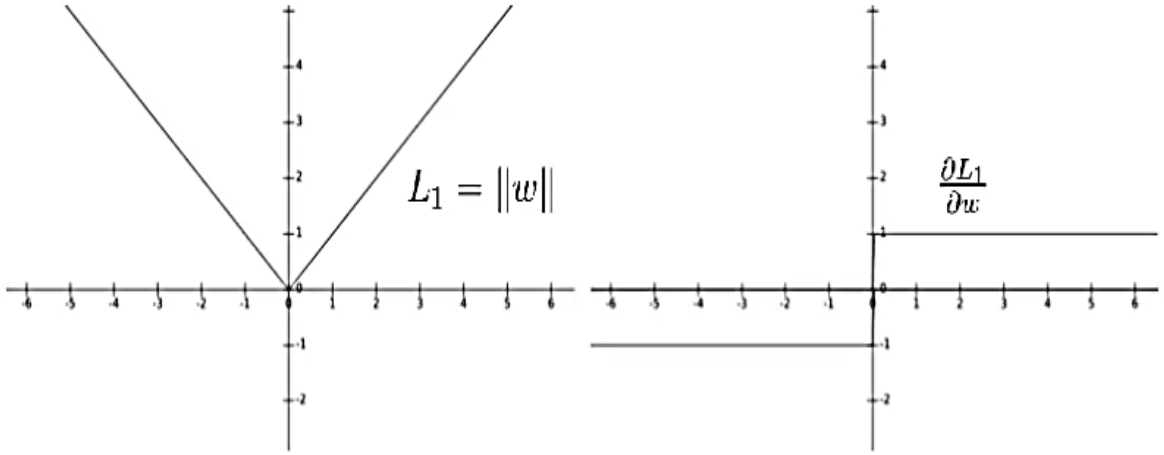
Sparsity

- A sparse autoencoder is simply an autoencoder whose training criterion involves a **sparsity penalty**.
- The intuition behind this method is that, for example, if a man claims to be an expert in mathematics, computer science, psychology, and classical music, he might be just learning some quite **shallow knowledge** in these subjects. If he only claims to be devoted to mathematics, we would like to anticipate some **useful insights** from him.
- It’s the same for autoencoders we’re training — fewer nodes activating while still keeping its performance would guarantee that the autoencoder is actually learning **latent representations** instead of redundant information in our input data.

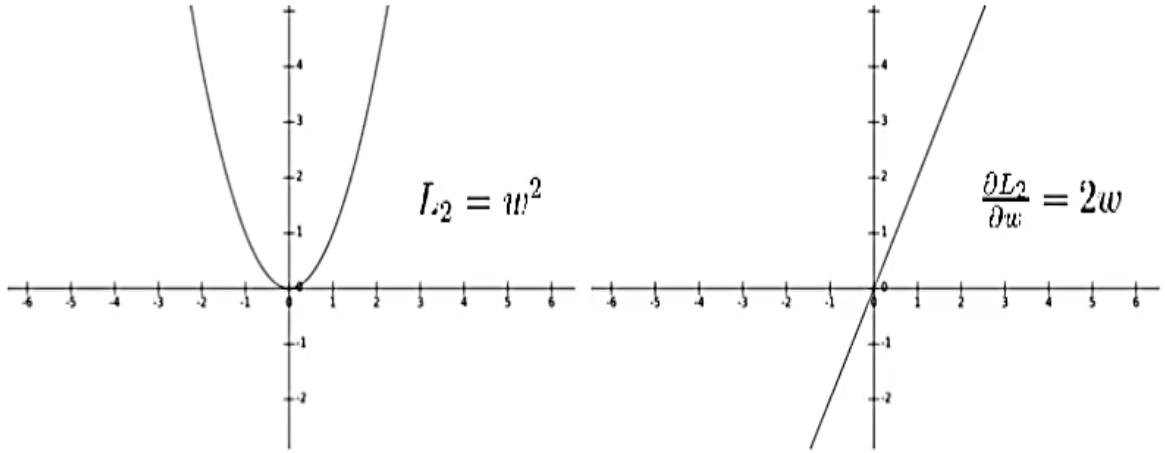
- Consider that we have two loss functions L1 and L2 which represent L1 regularization and L2 regularization respectively.

$$L_1 = \|w\|, L_2 = w^2$$

Gradient descent is always used in optimizing neural networks. If we plot these two loss functions and their derivatives, it looks like this:



L1 regularization and its derivative



L2 regularization and its derivative

- We can notice that for L1 regularization, the gradient is either 1 or -1 except when w=0, which means that L1 regularization will always move w towards zero with same step size (1 or -1) regardless of the value of w.
- And when w=0, the gradient becomes zero and no update will be made anymore. However, for L2 regularization things are different.
- L2 regularization will also move w towards zero but the step size becomes smaller and smaller which means that w will never reach zero.

This is the intuition behind L1 regularization's sparsity.

Shallow Autoencoder

- Input (784) → Hidden (128) → Latent (32) → Hidden (128) → Output (784)

Deep Autoencoder

- Input (784) → 512 → 256 → 128 → Latent (32) → 128 → 256 → 512 → Output (784)

Regularized Autoencoders

- Instead of relying on bottleneck size, **regularized autoencoders** introduce **constraints or penalties** to enforce meaningful feature learning

Sparse Autoencoder

- Adds a **sparsity constraint** (L1 penalty) on activations in the hidden layers.
- Only a few neurons fire per input → useful for **sparse representation**.

$$\mathcal{L} = \|X - \hat{X}\|^2 + \lambda \sum_i |h_i|$$

Denoising Autoencoder

A **Denoising Autoencoder** is a type of **regularized Autoencoder** that learns to reconstruct the **original clean input** from a **corrupted (noisy) version** of the input.

- Real-world data often contains noise.
- Denoising autoencoders help learn **robust representations** that capture the underlying structure of the data rather than memorizing the noise.
- Trained to reconstruct the input from a noisy version.
- Learns robust features that capture underlying structure, not noise.

Training with squared error criterion

$$||g(f(\hat{x})) - x||^2$$

- Noise regularization involves corrupting the input data (adding Gaussian noise or masking) and training the autoencoder to reconstruct the original clean input.

$$x_{\text{noisy}} = x + \text{noise}, \quad \text{train to minimize } \|x - \hat{x}\|^2$$

- Original Image: Electron microscope image of brain tissue.
- Mask: A grayscale or color image where white pixels represent cells, black represents background.

- Instead of training the autoencoder with X we corrupt the input X to get a noisy version X~, and train the model to map:X~→X.
- The latent code (latent representation, embedding, or bottleneck features) is the compressed vector produced by the encoder part of an Autoencoder.

Clean Input (X) → Add Noise → Noisy Input (X~)

↓ ↓

Encoder Encoder

↓ ↓

Latent Code Latent Code

↓ ↓

Decoder Decoder

↓ ↓

Reconstructed $\hat{X}$ (output) ← compare with original X

- The encoder compresses the input data (an image).
- The latent code is the output of the encoder, representing the most important features of the input.
- The decoder then tries to reconstruct the input from this latent code.
- Encoder: 784 → 128 → 64 → **32** ← latent code
- For a 28×28 image (784 features), encoder outputs a latent code of 32 features. So each image is now represented as a vector of length 32.
- The latent code is the core representation learned by an autoencoder. It's compact, meaningful, and useful for many tasks.

- **Loss Function:**The objective is to minimize reconstruction error.
- **Gaussian noise:** Add small random values to pixels.
- **Masking noise:** Randomly set some input values to zero.
- **Salt-and-pepper noise:** Randomly set some pixels to 0 or 1.

$$\mathcal{L} = \|X - \hat{X}_{\text{decoded from noisy } X'}\|^2$$

Denoising autoencoder (DAE) is an autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Linear(256, 64)
        )
        self.decoder = nn.Sequential(
            nn.Linear(64, 256),
            nn.ReLU(),
            nn.Linear(256, 784),
            nn.Sigmoid()
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

```
# Add Gaussian noise
def add_noise(inputs, noise_factor=0.3):
    noisy = inputs + noise_factor * torch.randn_like(inputs)
    return torch.clip(noisy, 0., 1.)
```

Deterministic encoder

A **deterministic encoder** is a function fθ(x) that maps input data x to a latent vector z without any randomness:

z=fθ(x)

θ - parameters (weights) of the encoder neural network.

Encoder compresses input x to a latent vector z

Decoder reconstructs x from z

There is **no sampling**, **no probability distribution**, and **no noise** involved in this encoding process.

Stochastic encoder

A **stochastic encoder** maps input data x to a **distribution over latent variables** z, instead of a single point.

- **Deterministic Encoder:** z=f(x)
- **Stochastic Encoder:** z∼q(z|x)

Even if you input the same image multiple times, the latent vector z will be different each time, because it's sampled from a distribution.

$$z \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

A latent vector is a numerical representation of input data in a compressed, abstract form

$$z \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

- z: the latent vector
- $\mu(x)$: the mean of the distribution for a given input x
- $\sigma 2(x)$: the variance for that input
- N: normal (Gaussian) distribution

So, rather than encoding an input x into a single deterministic latent vector, encode it into a probability distribution, and sample z from that distribution.

Latent vector

A latent vector (latent representation) is a compressed, abstract numerical representation of the original input data, capturing its essential features while discarding unnecessary details.

It's often used in autoencoders, where it acts as an input to generate new data samples or compress existing ones.

Compression: Latent vectors reduce the dimensionality of data, making it easier to work with and potentially removing noise.

Abstraction: They capture high-level features and patterns, allowing models to generate or reconstruct data based on these underlying characteristics.

Generative Models: In Generative Adversarial Networks (GANs), a latent vector is fed into the generator to produce new data instances.
Autoencoders: In autoencoders, the latent vector represents the compressed form of the input data after it has been processed by the encoder.
The decoder then reconstructs the data from this latent representation.
Latent Space: The space where these latent vectors reside is called the latent space.

Example: Imagine you have a dataset of faces.

A latent vector could represent the underlying features of those faces (shape of the nose, the curve of the lips, etc.) in a compact numerical form.

This latent vector can then be used to generate new, unique faces that share similar characteristics.

Encoder and decoder are both feed forward networks, they can individually benefit from depth or as a whole.

Encoding: The process of converting data into a latent vector.

Decoding: The process of reconstructing data from a latent vector.

Dimensionality Reduction: The act of reducing the number of variables needed to represent data.

Feature Extraction: Identifying and representing the most important characteristics of data.

The latent vectors are a powerful tool for working with complex data, allowing for efficient representation, generation, and manipulation of data in various machine learning applications.

Mainly used in:

- **Dimensionality reduction-makes computation faster.**
- **Feature extraction.**
- **Data generation (decoder can reconstruct images from latent space)**

Predictive Sparse Decomposition

Predictive Sparse Decomposition (PSD) is a feature learning method that extends sparse coding by introducing a predictive model that can quickly infer sparse codes for new data, without the need to solve a costly optimization problem every time. Training Proceeds by minimizing

$$||x - g(x)||^2 + \lambda|h|_1 + \gamma||h - f(x)||^2$$

x	Input data
g(x)	Decoder output (reconstructed input)
f(x)	Encoder output (latent code)
h	Target latent representation
h1	Possibly an intermediate or first hidden layer (regularized)
λ lambda, γ gamma	Regularization hyper parameters (weights)

Reconstruction Loss:

- $\|x-g(x)\|^2$: Measures how well the autoencoder reconstructs input x.
- Lower is better.

$$\lambda\|h_1\|$$

Regularization Term: $\lambda\|h1\|$Penalizes large weights or activations in layer $h1$

Encourages sparsity or smoothness depending on norm used (L2 or L1).

Latent Alignment Term:

$$\gamma\|h - f(x)\|^2$$

Encourages the learned latent representation $f(x)$ to be close to some desired representation h.

- During training, h is controlled by the optimization algorithm.
- The training alternates between minimization with respect to h and minimization respect to the model parameters.
- The interactive optimization is used only during training.
- The parametric encoder f is used to compute the learned feature when the model is deployed.
- PSD models maybe stacked and used to initialize a deep network to be trained with another criterion.

Applications

Dimensionality Reduction: Feature extraction-Reduce high-dimensional data (like images) to a compact latent vector.

Denoising: Denoising autoencoders-Remove noise from images, audio, or signals by learning clean representations.

Anomaly Detection: Fraud, intrusion, defect detection, Train on normal data → large reconstruction error indicates anomalies.

Data Compression: Use encoder-decoder structure for compressing and reconstructing data.

Pretraining: Weight initialization-Use unsupervised pretraining to initialize deep networks, especially useful in low-data settings.

Generative Adversarial Network (GAN)

- GANs belong to the formidable realm of neural networks and are employed for generative modeling through unsupervised learning, as highlighted by key references such as Bengio (2014) and Babcock and Bali (2021).
- At its core, a GAN consists of a dynamic interplay between two neural network models engaged in a competitive dance, capable of comprehending, capturing, and replicating variations within a dataset.
- These two networks assume the roles of the generator and the discriminator, each with its own unique function.
- The generator, typically implemented as a CNN, takes random noise as input and crafts an image that aligns with the desired domain.
- In contrast, the discriminator's role is to discern whether the generated data, such as an image, is authentic or not. In other words, it evaluates whether the output genuinely belongs to the intended distribution or not.

- This framework not only facilitates the generation of synthetic data but also enhances the performance of learning models as a data augmentation technique, reducing generalization errors.
- It achieves this by generating new, artificial yet plausible examples from the input problem domain on which the model is trained.
- In theory, GANs operate within a framework rooted in game theory.
- Within this frame work, a generating neural network engages in a strategic competition with an adversarial network.
- The role of the generator network is to produce samples directly, while its counterpart, the discriminative network, strives to distinguish between samples originating from the training data and those generated by the generator.

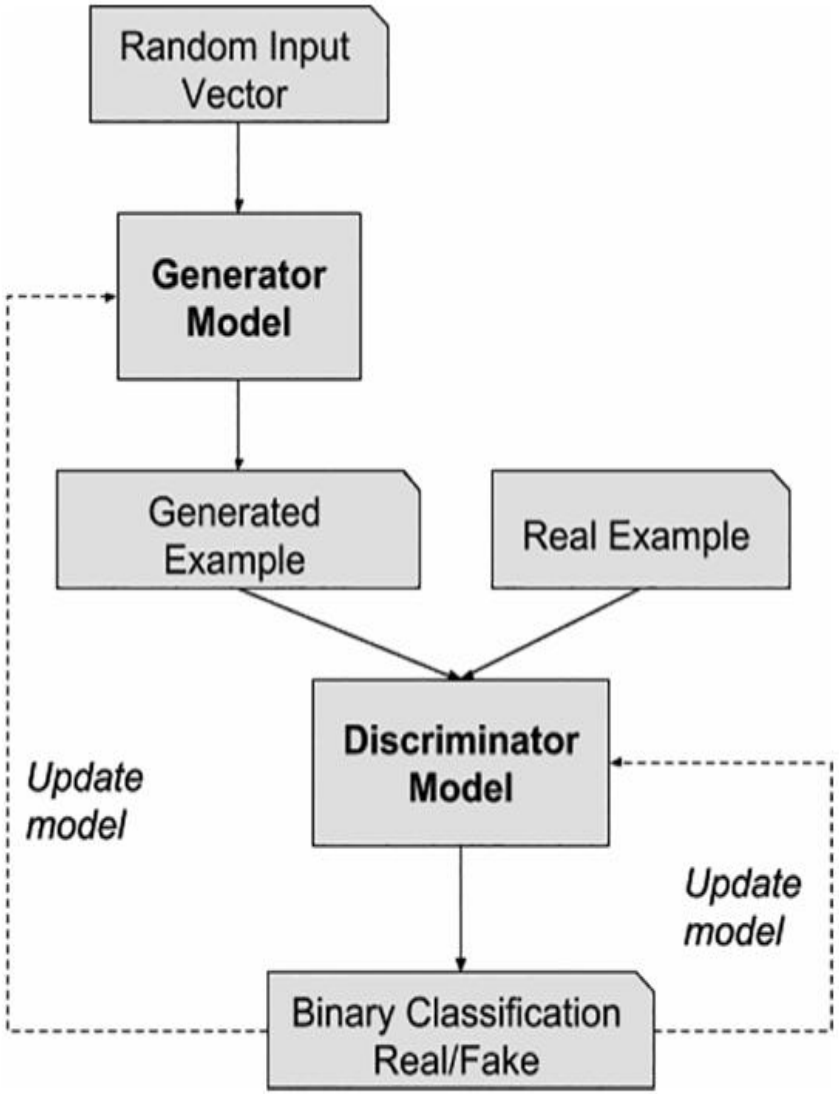
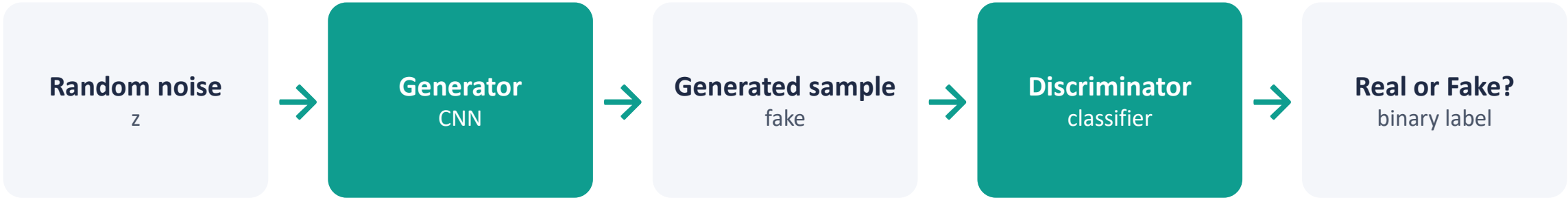


FIGURE 2.21 Composition of a GAN.

Generative Adversarial Networks (GANs)

- Neural networks for generative modeling via unsupervised learning
- Two networks compete: a generator and a discriminator
- Game-theory framework: generator creates samples, discriminator tells real from fake
- Also used for data augmentation, reducing generalization error



Real samples from the training data also feed the discriminator

GAN Components

Generator (generative model)

- Input: random fixed-length vector from a Gaussian distribution
- Output: a sample in the problem domain
- Vector space = compressed representation with latent variables
- The model we keep after training

Discriminator (discriminative model)

- Input: real or generated example
- Predicts binary label: real or fake
- A standard classification model
- Role ends after training; generator layers can be reused for transfer learning

GANs vs Autoencoders

Autoencoders

- Results often closely aligned with the training data
- Learn compressed representations (encoder → latent → decoder)

GANs

- Generate high-quality, realistic content
- Adversarial setup teaches the generator to mimic training patterns
- Capture the underlying distribution of the original dataset
- Suited to text and image generation

- GANs introduced an adversarial approach involving two neural networks: Generator and Discriminator
- The generator creates synthetic data, while the discriminator attempts to distinguish real data from generated data.

2.9.1 The Generative Model

In the realm of generative models, a pivotal role is played by the generative model itself. This model takes as its input a random vector of fixed length, a vector drawn randomly from a Gaussian distribution, and then works its magic to generate a sample within the defined domain (as illustrated in Figure 2.22). It's important to note that this multidimensional vector space, arising from the generative process, eventually houses points that correspond to the problem domain. In essence, this space serves as a compressed representation of the data distribution.

Within this vector space lie latent (hidden) variables, which carry significance for the domain but remain elusive to direct observation.

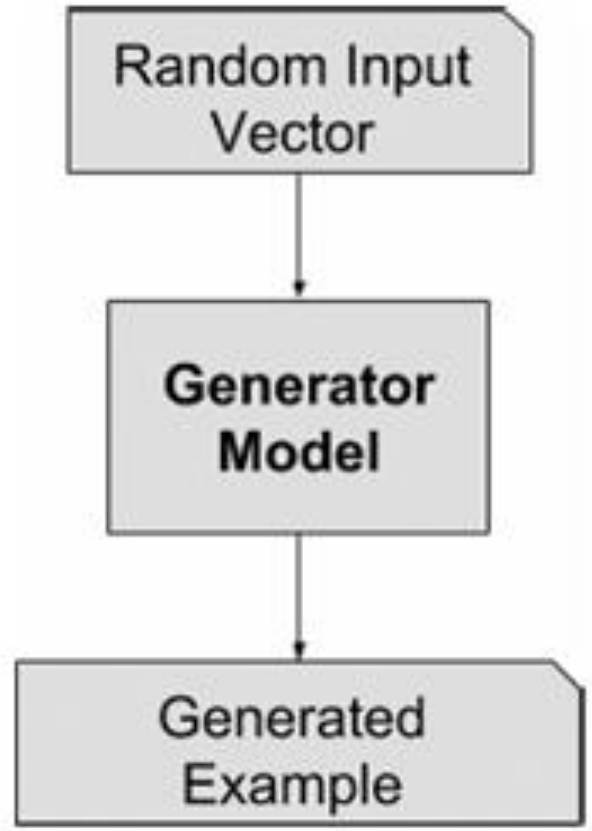


FIGURE 2.22 The generative component of a GAN.

2.9.2 The Discriminative Model

In contrast, the discriminative model takes an example from the domain, whether it's real or generated, and plays a pivotal role in predicting a binary class label: either real or fake (as depicted in Figure 2.23). Real examples are drawn from the training *dataset*, while the generated instances are the handiwork of the generator model. The discriminator functions as a conventional and well-understood classification model.

Following the training phase, the discriminator model's role concludes, as our primary interest lies in the generator model. It's worth noting that the generator, having honed its ability to extract *features* effectively from domain examples, can sometimes find further utility. In particular, its *feature* extraction layers can be employed in transfer learning applications that involve similar or identical input data.

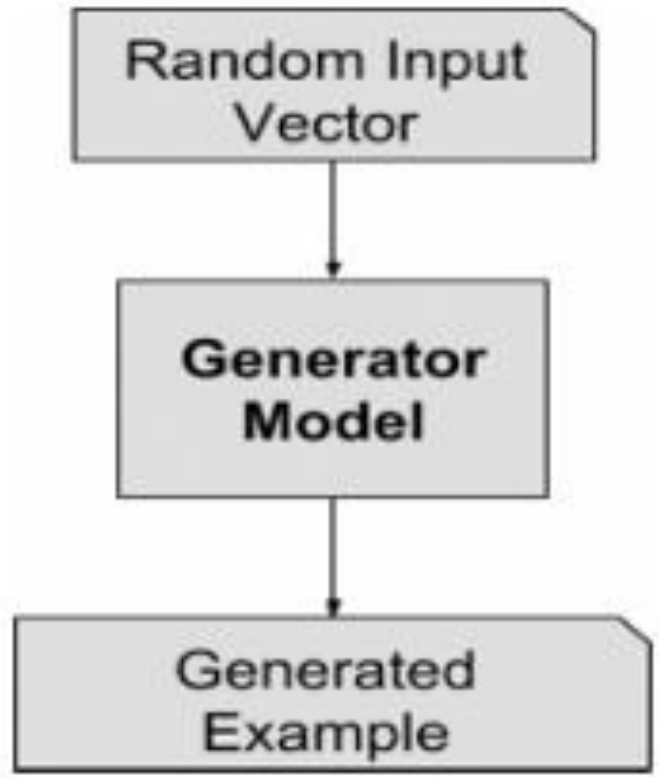
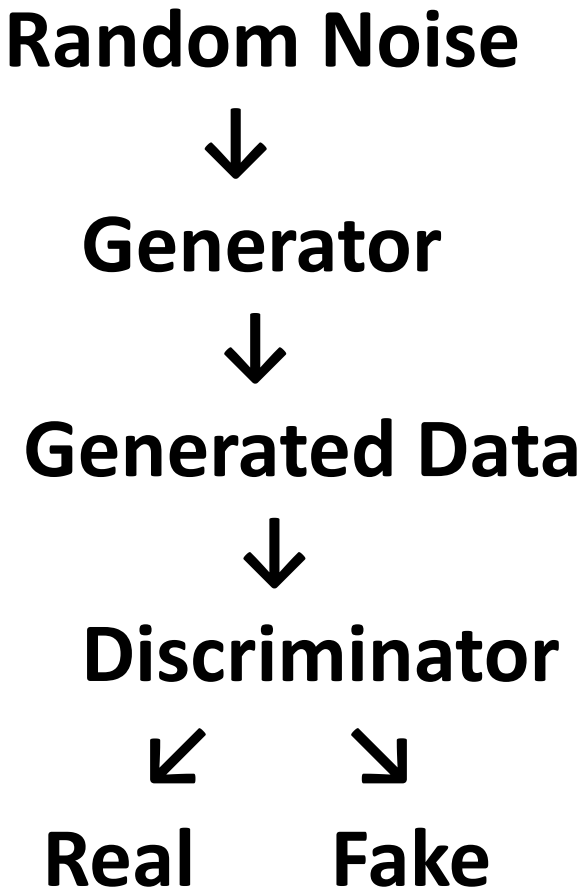


FIGURE 2.23 The discriminative component of a GAN.


Random Noise
↓
Generator
↓
Generated Data
↓
Discriminator
↙ ↘
Real Fake

Transformers

- Transformers rely on four major components working together: self-attention, multi-head attention, positional encoding, and feed-forward networks.
- Together, these components allow modern AI systems to understand relationships, context, meaning, and structure far more effectively than older AI architectures.

Self-attention

- The most important idea in the Transformer architecture is self-attention.
- Self-attention allows every word in a sentence to look at every other word and decide which ones matter most. That is the heart of the attention mechanism.

"The cat sat on the mat."

- When processing the word "cat," the model does not look only at nearby words.
- It evaluates the entire sentence simultaneously.

Transformer asks three questions for every word.

- **Query:** "What information am I looking for?"
- **Key:** "What kind of information do I offer?"
- **Value:** "What actual information do I carry?"
- A matchmaking system between words.
- For the word "cat,"
- the Query asks what relationships matter, the model compares the query against the Keys of every other word, and strong matches receive closer attention.

"cat" might strongly attend to "sat" (action relationship) and "mat" (location relationship), and weakly attend to smaller function words like "the" and "on," which still matter but less strongly.

The result is that the model creates a richer understanding of "cat," not as an isolated word, but as "the cat that sat on the mat."

Self-attention solved several major problems at once:

- Every word can directly connect to every other word
- Long-distance relationships are preserved
- Processing happens in parallel
- Context understanding improves dramatically.

Multi-head attention

- Instead of using one attention system, Transformers run multiple attention mechanisms simultaneously.
- These are called attention heads.
- You can think of them as multiple specialists analyzing the same sentence from different perspectives.
- One head might focus on grammar: subjects, verbs, and sentence structure.
- Another focuses on semantic meaning: "cat" as an animal, "mat" as an object.
- Another focuses on position: which words appear earlier or later.
- Another focuses on references: "it" referring to "cat."

Positional encoding

There was one major challenge with parallel processing. If Transformers process all words simultaneously, how do they know word order?

Consider: "Dog bites man." and "Man bites dog."

- These contain the same words but completely different meanings.
- This is where **positional encoding** comes in.
- Transformers add position information to every word before processing begins.
- Word 1 receives one positional signal
- word 2 receives another, and so on.
- This allows the model to preserve sequence information while still processing everything in parallel.

Feed-forward network

- After attention gathers context, the Transformer still needs to refine and strengthen its understanding.
- Polishing the interpretation.
- Attention identifies relationships.
- Feed-forward layers help transform those relationships into richer internal representations.
- The model repeatedly improves its understanding of what each word represents in context.
- This refinement process helps Transformers become better at prediction, reasoning, classification, generation, and summarization.
- Every layer adds more contextual depth.

Three Reasons Transformers Beat RNNs

1. **Parallel Processing Makes Transformers Much Faster**
2. **Transformers Understand Long-Range Context Better**
3. **Transformers Scale Extremely Well**

Modern Generative AI

Modern Generative AI can work with multiple modalities:

- **Text→Text**
- **Text→Image**
- **Text→Audio**
- **Text→Video**
- **Text + Image + Audio + Video → Integrated**

Attention

Attention = Weighted embeddings

-29.0	-18.5	-30.9	-8.9	2.8
4.4	30.8	16.4	29.5	-7.0
16.0	-9.2	-16.4	-1.5	2.1
46.5	10.3	67.3	47.2	16.9

-84.5
74.0
-9.0
188.1

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

I	-2	0.3	1.1	3.2
can	-1.8	3	-0.9	1
bear	-1.7	0.9	-0.9	3.7
the	0.6	-2	0.1	-3.2
pain	-0.4	1	-0.3	-2.4

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

Attention Terminology

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

These vectors in this operation are called as Queries(Q)

bear	-1.7	0.9	-0.9	3.7

Attention Terminology

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

bear	-1.7	0.9	-0.9	3.7

These vectors in this operation are called as Queries(Q)

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

These vectors in this operation are called as keys(K)

Attention Scores

This is the dot product between the words. These are known as **attention scores**

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

These vectors in this operation are called as Queries(Q)

bear	-1.7	0.9	-0.9	3.7

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

These vectors in this operation are called as keys(K)

Attention Terminology

I	can	bear	the	pain
-29.0	-18.5	-30.9	-8.9	2.8
4.4	30.8	16.4	29.5	-7.0
16.0	-9.2	-16.4	-1.5	2.1
46.5	10.3	67.3	47.2	16.9

We multiply weights again with the Vector(V) embeddings.

These vectors in this operation are called as Queries(Q)

bear	-1.7	0.9	-0.9	3.7

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

These vectors in this operation are called as keys(K)

Attention Terminology

I	can	bear	the	pain
-29.0	-18.5	-30.9	-8.9	2.8
4.4	30.8	16.4	29.5	-7.0
16.0	-9.2	-16.4	-1.5	2.1
46.5	10.3	67.3	47.2	16.9

We multiply weights again with the Vector(V) embeddings.

-84.5
74.0
-9.0
188.1

These vectors in this operation are called as Queries(Q)

These are called scores. Scores are normalized and their sum is made as 1.

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

bear	-1.7	0.9	-0.9	3.7

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

These vectors in this operation are called as keys(K)

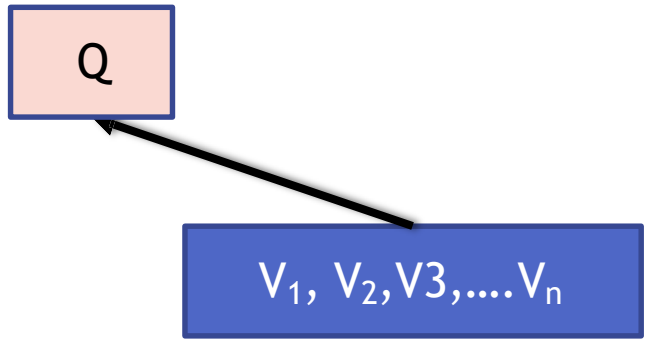
Attention Terminology

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4

$V_1, V_2, V3, \ldots V_n$

Attention Terminology

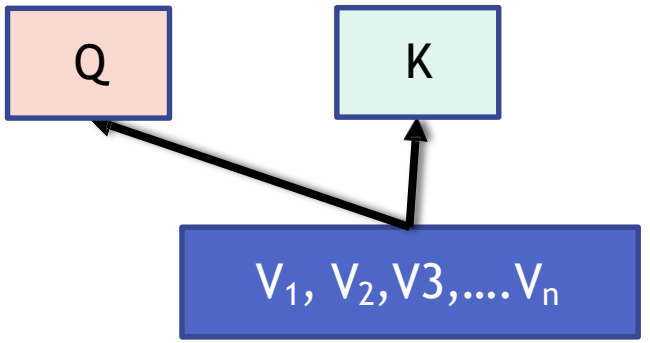
bear	-1.7	0.9	-0.9	3.7



Attention Terminology

bear	-1.7	0.9	-0.9	3.7

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4



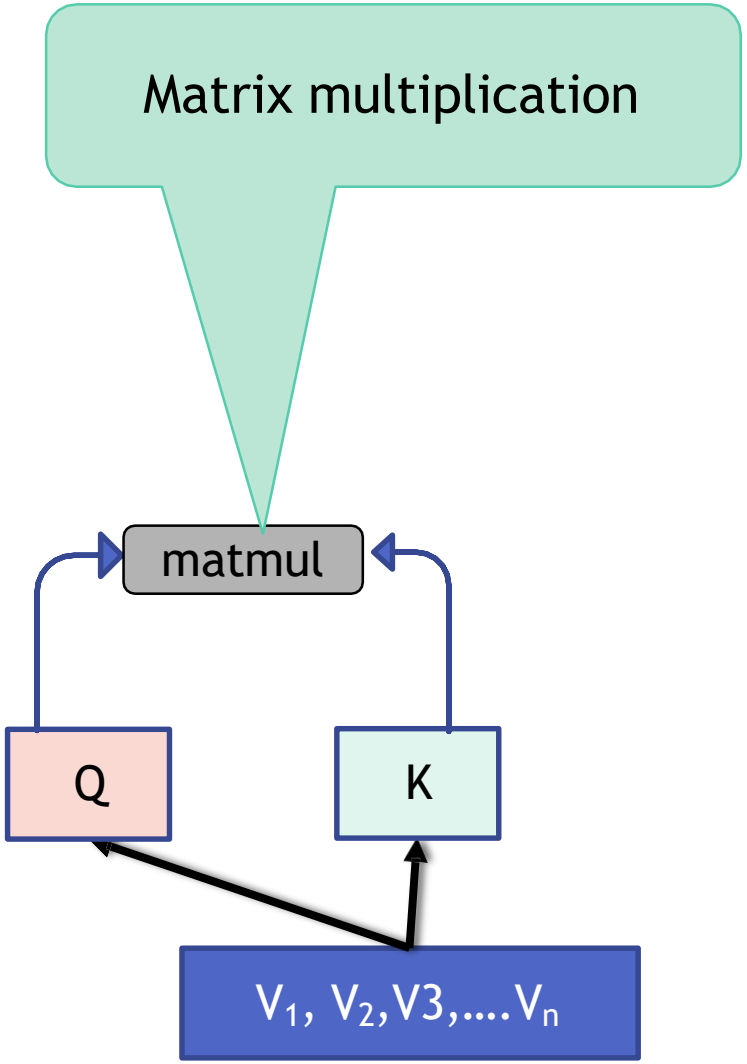
Attention Terminology

Attention scores

14.52	10.27	18.20	-14.75	-7.03

bear	-1.7	0.9	-0.9	3.7

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4



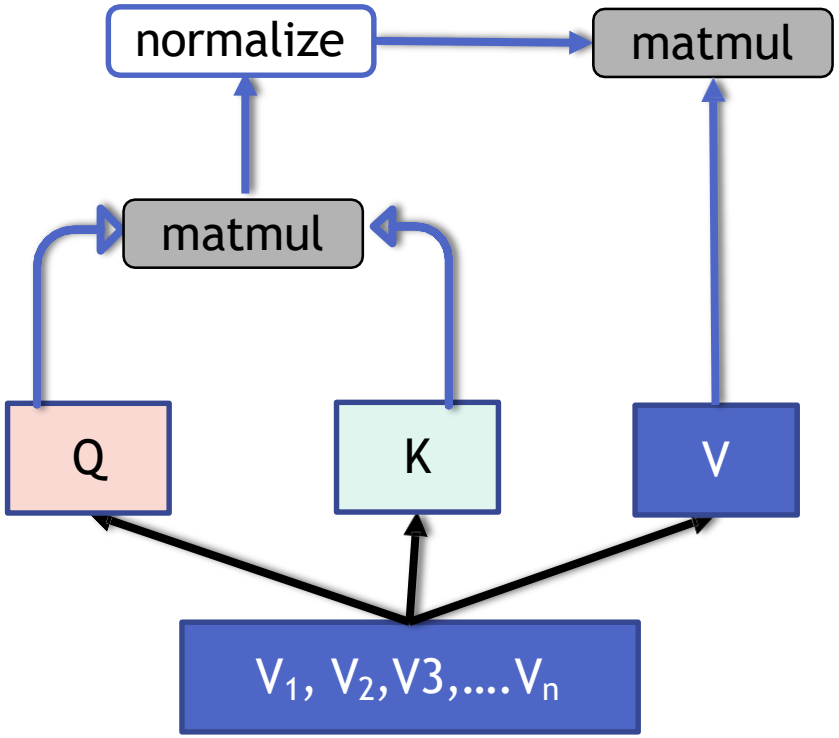
Attention Terminology

-29.0	-18.5	-30.9	-8.9	2.8
4.4	30.8	16.4	29.5	-7.0
16.0	-9.2	-16.4	-1.5	2.1
46.5	10.3	67.3	47.2	16.9

14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

bear	-1.7	0.9	-0.9	3.7

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4



Only one vector Y_k calculation is shown
We can do the same for all ‘n’ vectors

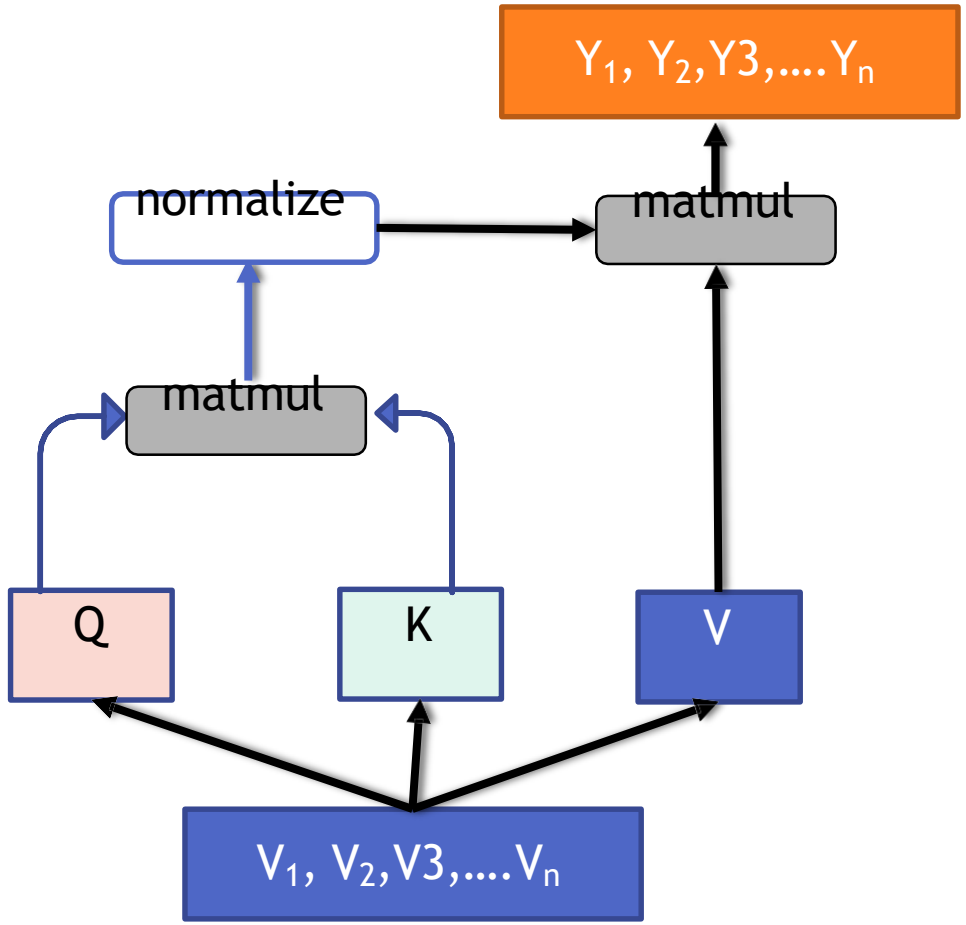
-29.0	-18.5	-30.9	-8.9	2.8
4.4	30.8	16.4	29.5	-7.0
16.0	-9.2	-16.4	-1.5	2.1
46.5	10.3	67.3	47.2	16.9

-84.5
74.0
-9.0
188.1

bear	-1.7	0.9	-0.9	3.7

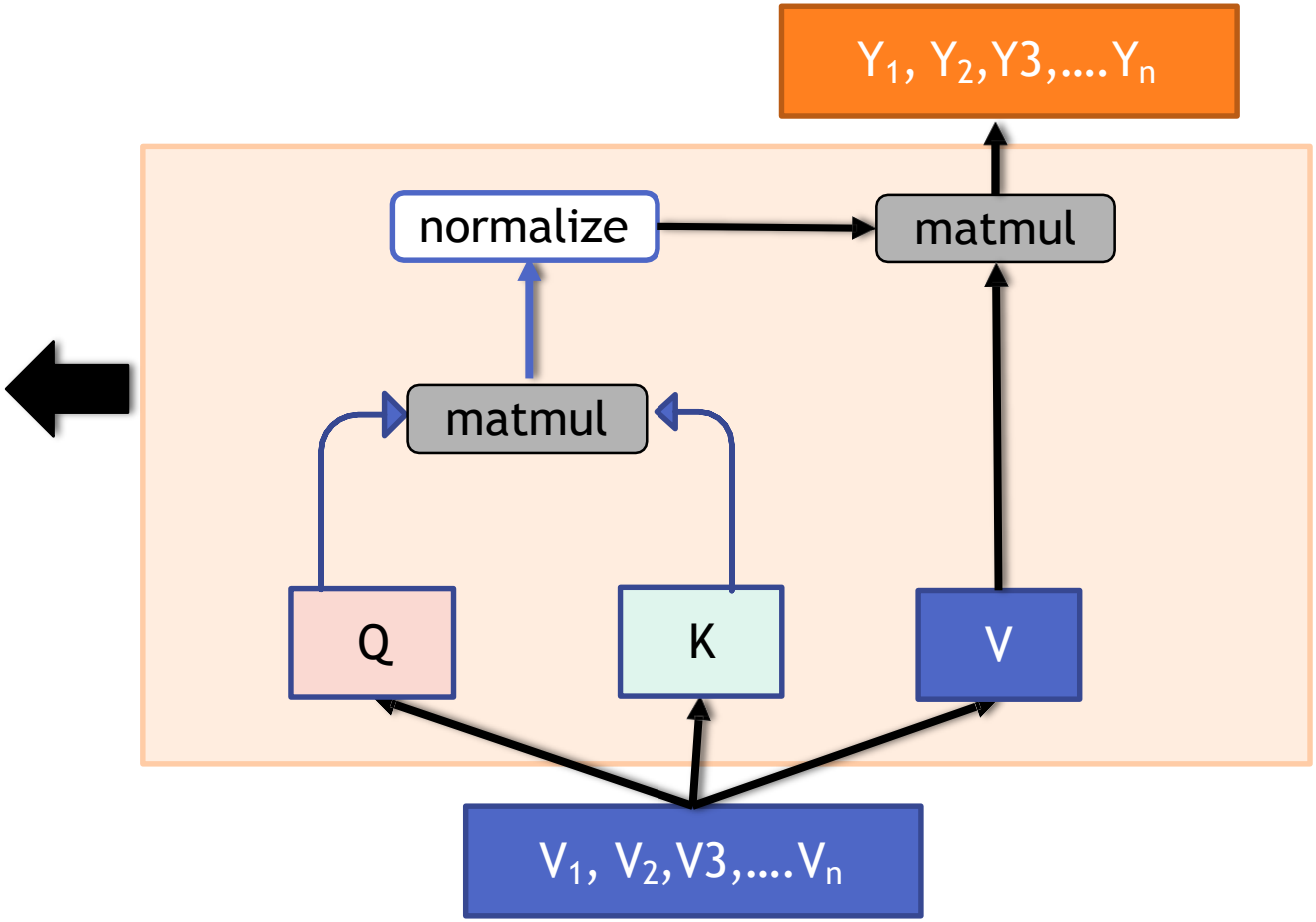
14.52	**10.27**	**18.20**	**-14.75**	**-7.03**

I	can	bear	the	pain
-2	-1.8	-1.7	0.6	-0.4
0.3	3	0.9	-2	1
1.1	-0.9	-0.9	0.1	-0.3
3.2	1	3.7	-3.2	-2.4



Attention - Block

Attention block, more specially self-attention block



- **Attention** is the key mechanism that allows Transformers to **focus on relevant parts** of the input when processing each token. It helps the model learn **relationships between words**, regardless of their distance in the sentence.
- Attention tells the model what to look at when making decisions for each word."Instead of processing words one by one (like RNNs), attention allows every word to interact with every other word in the sentence, in parallel.
- **"The cat sat on the mat.“** When processing the word "sat", attention might focus on: "cat" (subject) "mat" (object)
- This helps the model understand contextual relationships.
- Each input token is transformed into three vectors:Query (Q)Key (K)Value (V)
- Attention
  - Is nothing but weighted embeddings only
  - Smart embeddings
  - Advanced version of word2vec
  - More contextualized embeddings
  - With a different matrix calculation formula.

Attention Score:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

QK^T : Measures similarity between queries and keys.

d_k : Scaling factor (prevents large dot-product values).

dk is the dimension (number of elements) in each Key vector.

Softmax: Converts scores into probabilities.

Multiply by V: Gets a weighted sum of the values.

Encoder Layer:

Self-attention → Focuses on all input tokens.

Decoder Layer:

Masked self-attention → Looks only at past outputs.

Cross-attention → Focuses on encoder outputs for translation.

Attention steps

Step	Action
1.	Each word creates Q, K, V vectors
2.	Compute attention scores ($Q \times K^t$)
3.	Apply softmax to get attention weights
4.	Use weights to combine value vectors (V)
5.	Use this as input for next layers

Attention Type	**Purpose**	**Usage**
Self-Attention	**Each word attends to every other word in the same sentence**	**Used in both encoder and decoder**
Cross-Attention	**Decoder attends to encoder outputs**	**Used in encoder-decoder models (e.g., translation)**
Masked Self-Attention	**Prevents a token from seeing future tokens**	**Used in decoder during training**

Code - Self Attention Practice

```
embeddings_example1 = {
    "Bank": np.array([0.1, 0.2, 0.3]),
    "Bear": np.array([0.4, 0.1, 0.6]),
    "Row": np.array([0.3, 0.5, 0.2]),
    "River": np.array([0.2, 0.3, 0.5]),
    "Seeds": np.array([0.3, 0.1, 0.4]),
    "Summer": np.array([0.6, 0.2, 0.1]),
}
```

```
embeddings_example2 = {
    "Bank": np.array([0.1, 0.2, 0.3]),
    "Bear": np.array([0.4, 0.1, 0.6]),
    "Row": np.array([0.3, 0.5, 0.2]),
    "Investors": np.array([0.7, 0.3, 0.4]),
    "Market": np.array([0.2, 0.8, 0.5]),
    "Dispute": np.array([0.4, 0.6, 0.7]),
}
```

Code - Self Attention Practice

```python
def calculate_attention(embeddings):
    words = list(embeddings.keys())
    embedding_matrix = np.array(list(embeddings.values()))

    # Use the same embeddings for Q, K, and V in self-attention
    Q = embedding_matrix  # Query
    K = embedding_matrix  # Key
    V = embedding_matrix  # Value

    # Compute self-attention
    attention_weights, attention_output = scaled_dot_product_attention(Q, K, V)

    return words, attention_weights, attention_output
```

Code - Self Attention Practice

```
# Calculate Attention for Example 1
print("Example 1: Riverbank, Produce, Arrangement")
words1, attention_weights1, output1 = calculate_attention(embeddings_example1)


#print("Words:", words1)
#print("Attention Weights:\n", attention_weights1)
#print("Attention Output (weighted sum of values):\n", output1)
for i in range(len(words1)):
  print(f"Attention values for {words1[i]}: {output1[i]}")
```

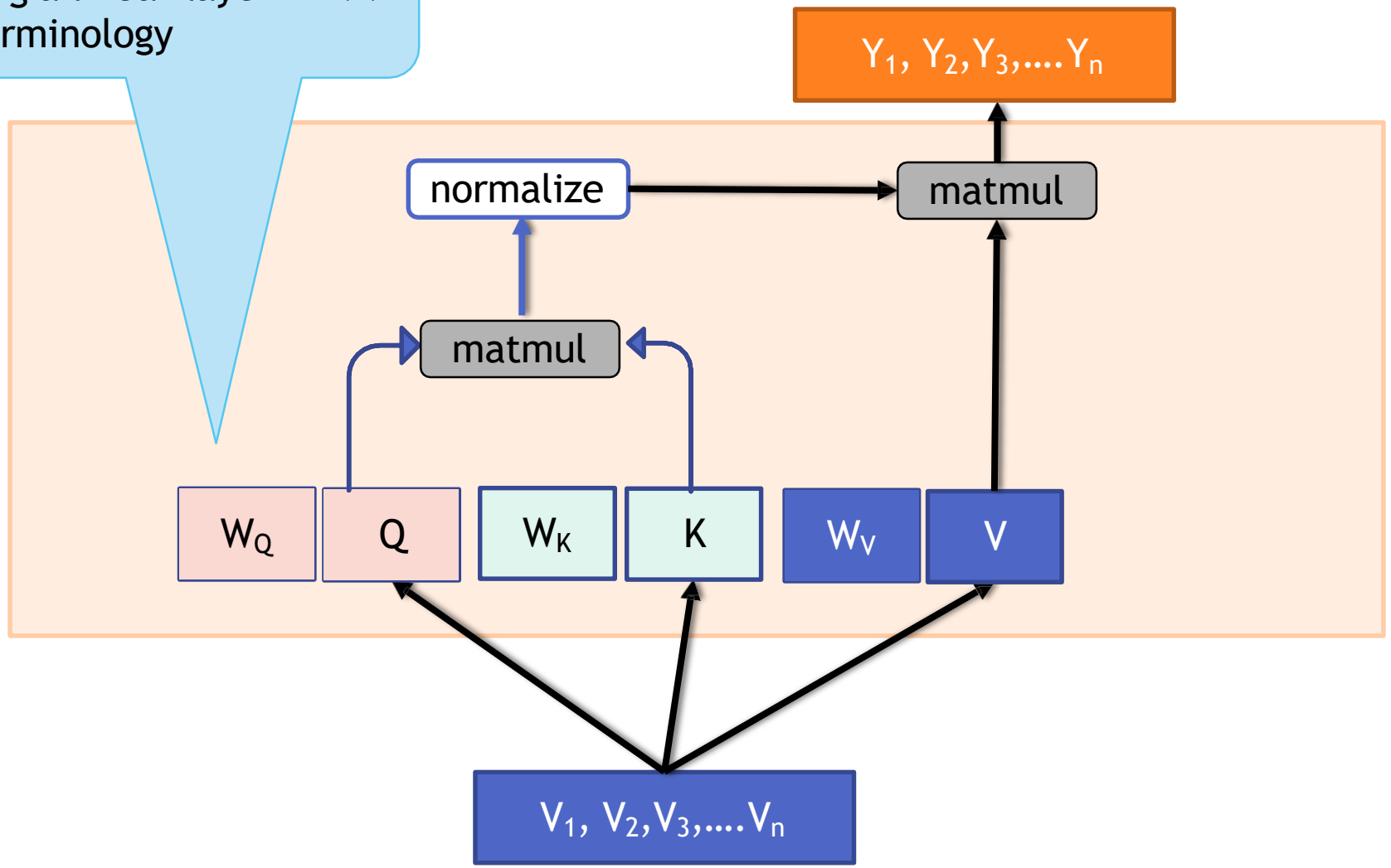
Code - Self Attention Practice

```
Example 1: Riverbank, Produce, Arrangement
Attention values for Bank: [0.31606865 0.23354518 0.35337653]
Attention values for Bear: [0.31895917 0.2299816  0.35756556]
Attention values for Row: [0.31871048 0.23700782 0.34884834]
Attention values for River: [0.31622049 0.23322745 0.35566971]
Attention values for Seeds: [0.31858798 0.23140021 0.35464133]
Attention values for Summer: [0.32427906 0.23355012 0.34721524]


Example 2: Financial Institution, Stock Market, Dispute
Attention values for Bank: [0.3522092  0.42391046 0.45617792]
Attention values for Bear: [0.36013129 0.41981272 0.46204445]
Attention values for Row: [0.35407018 0.4324855  0.45640699]
Attention values for Investors: [0.36436285 0.42392458 0.46049966]
Attention values for Market: [0.3512429  0.44458537 0.46232254]
Attention values for Dispute: [0.35728847 0.43720338 0.46560293]
```

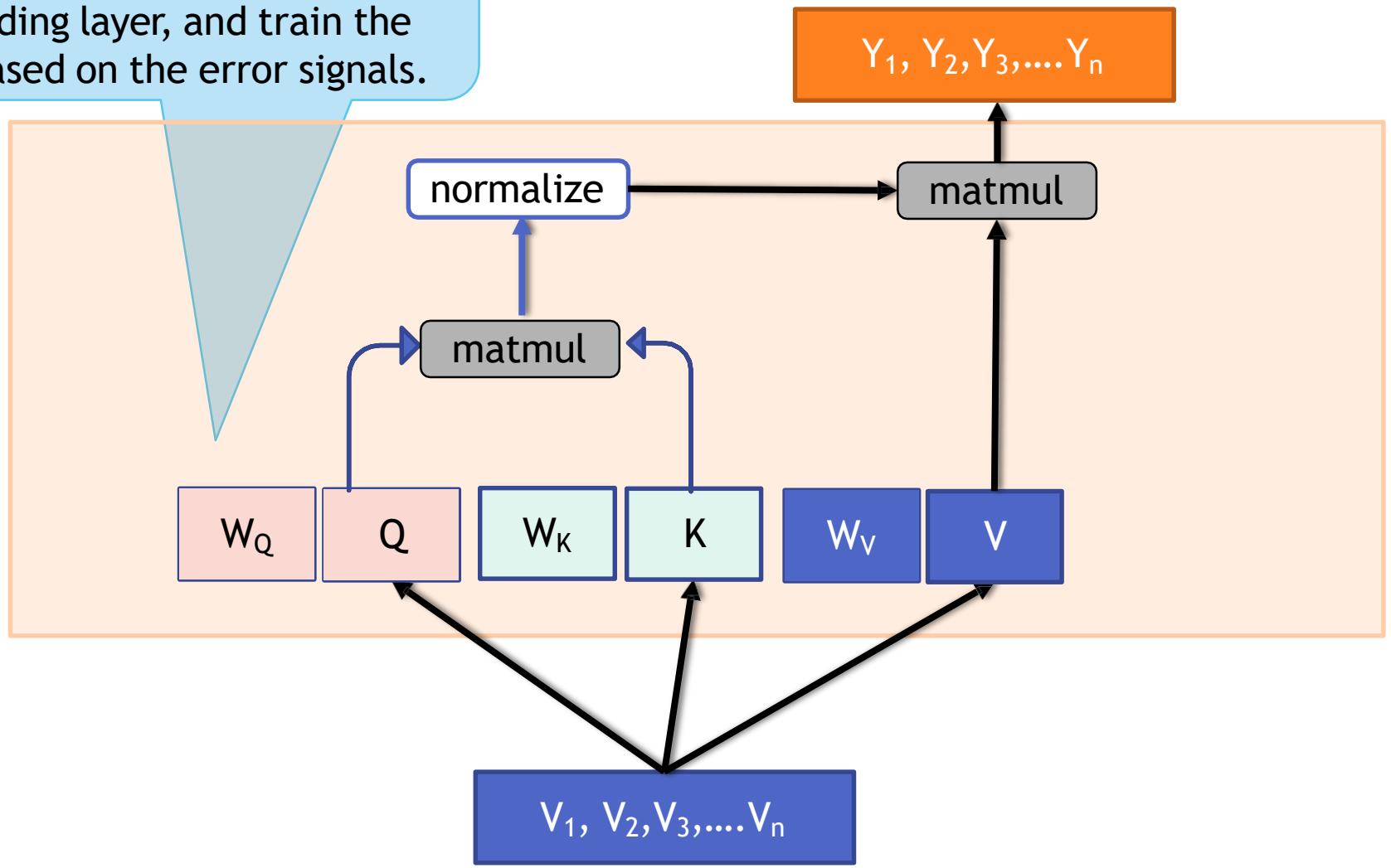
Add Trainable Parameters

Multiplying with weight matrix is same as adding a linear layer in NN terminology



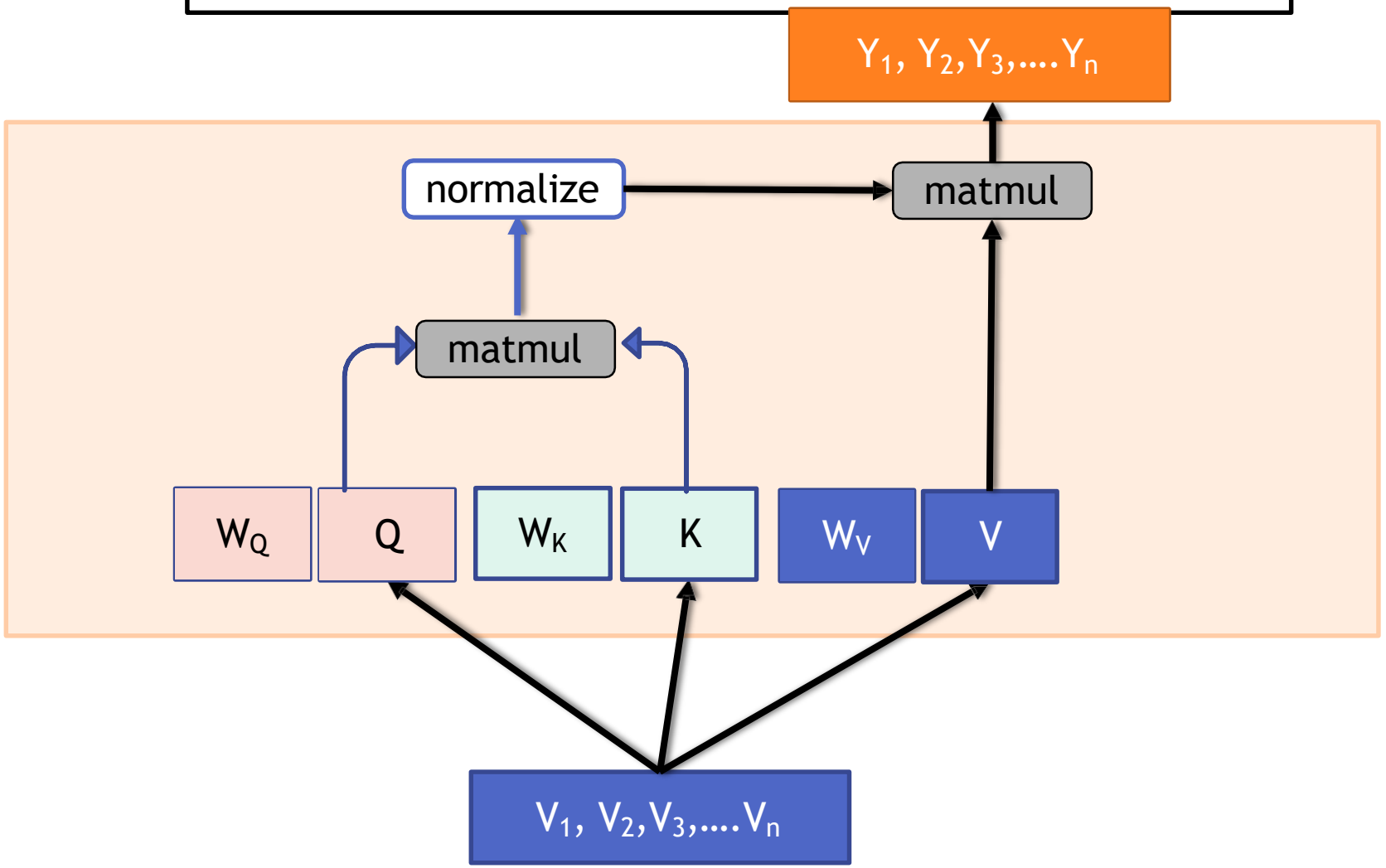
Add Trainable Parameters

While training the model, we can consider this as attention block as a linear embedding layer, and train the parameters based on the error signals.



Attention formula

$$Attention(Q, K, V) = softmax(QK^T)\mathrm{V}$$



Multi-head attention

Single Context

- The **bank** offers various credit cards for diverse customer needs.
- The river **bank** showcases stunning flowers in full, colorful bloom.

- Savings bank vs River bank - We can distinguish both by using attention mechanism.

Several parallel contexts

- The **bank** offers various credit cards for diverse customer needs.
- **Bank** shares experienced a stock price decline during this quarter.
- The **bank** sponsors a football tournament to expand its customer base.

- The river **bank** showcases stunning flowers in full, colorful bloom.
- Ancient cities formed along river **banks** during human evolutionary history.
- The river **bank** is steep, so we must be careful.

- Savings bank vs River bank - We can distinguish both by using attention mechanism.
- But, what if, bank itself has several sub contexts like bank credit cards, bank shares, bank sponsorship?
- How can we capture several parallel contexts ?

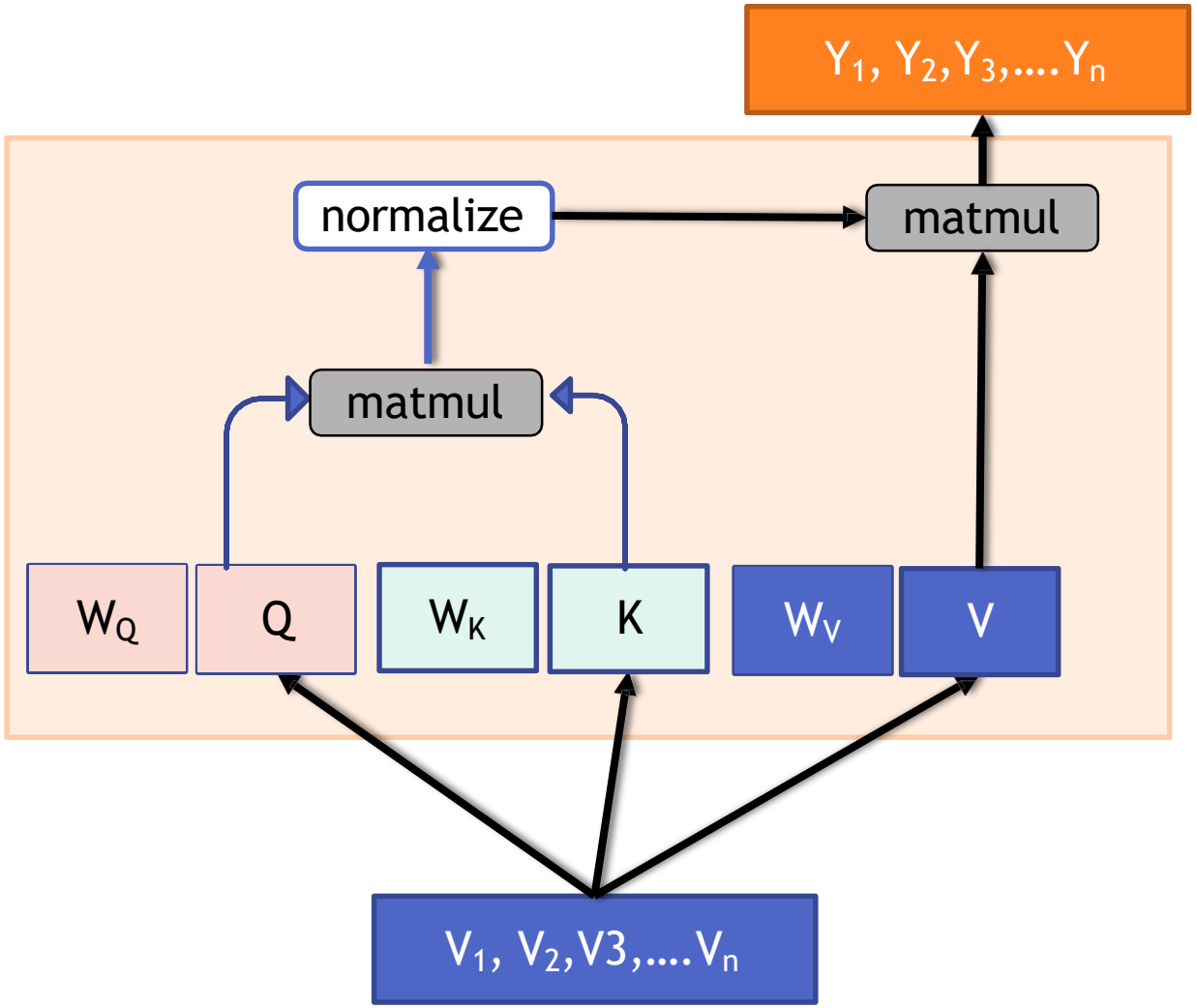
Multi-head attention

- The **bank** offers various credit cards for diverse customer needs.
- **Bank** shares experienced a stock price decline during this quarter.
- The **bank** sponsors a football tournament to expand its customer base.

- The river **bank** showcases stunning flowers in full, colorful bloom.
- Ancient cities formed along river **banks** during human evolutionary history.
- The river **bank** is steep, so we must be careful.

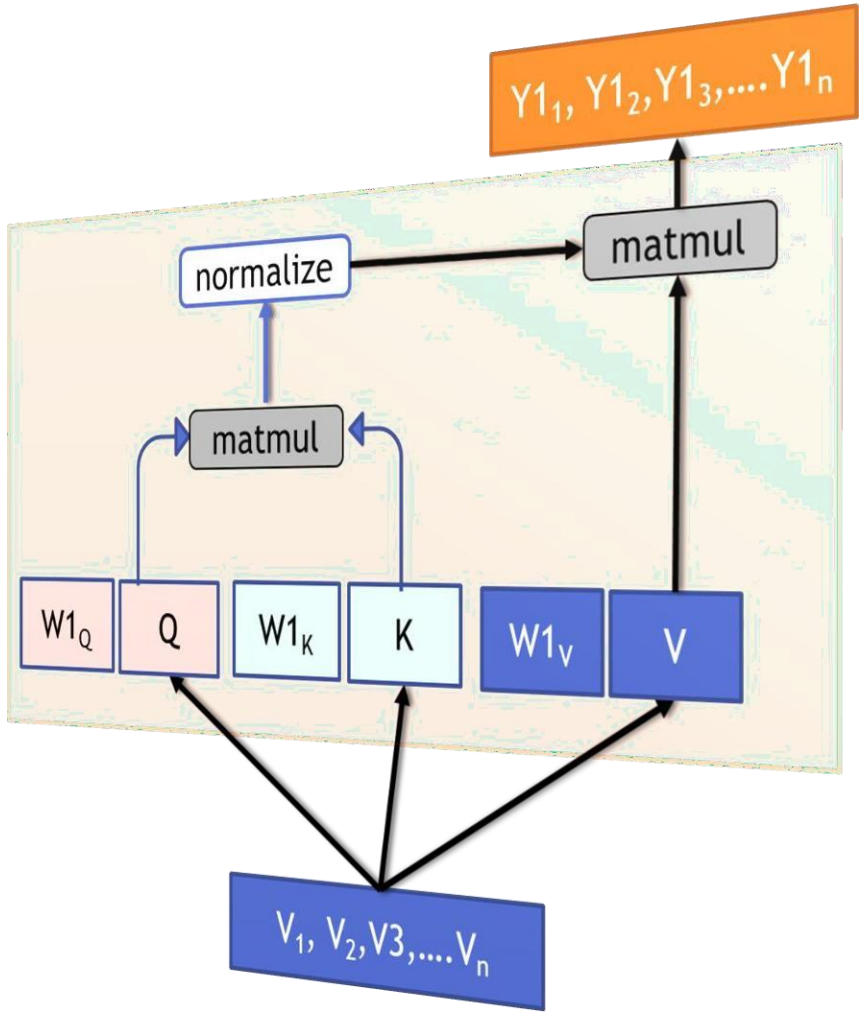
- But, what if, bank itself has several sub contexts like bank credit cards, bank shares, bank sponsorship?
- How can we capture several parallel contexts ? – We can use multiple self attention blocks in parallel.

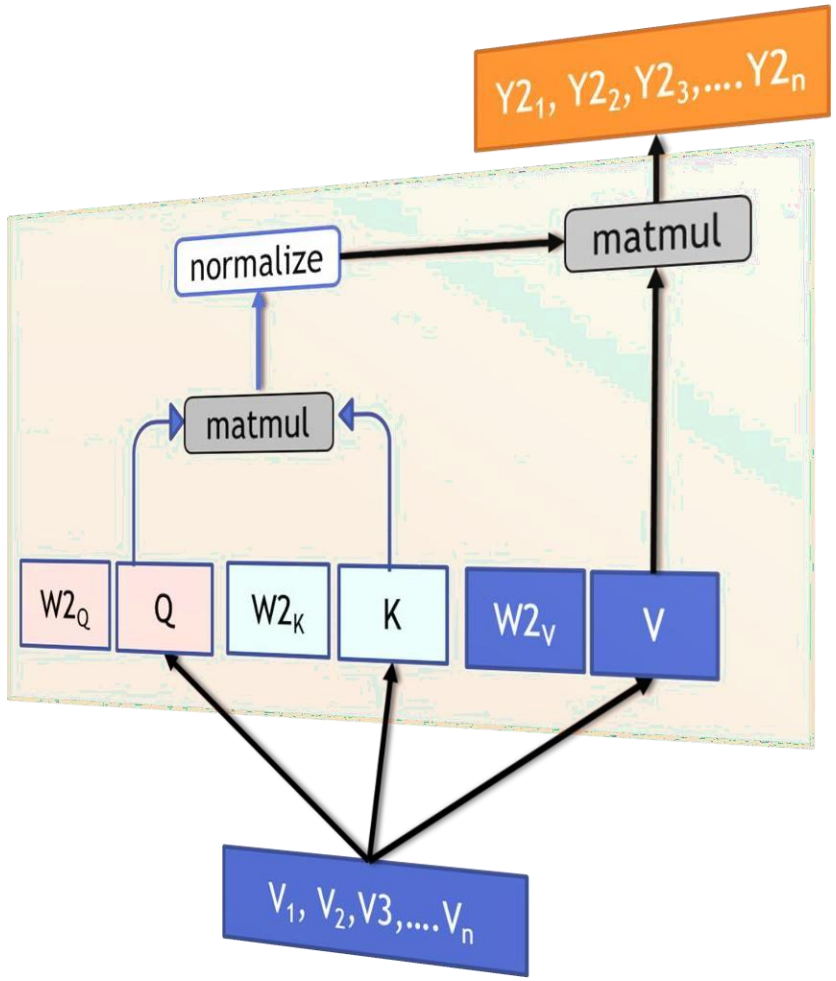
Single Head Attention.



- Until now, we have discussed a single attention stack either with single block or multiple attention blocks.
- At the end we will have a single set of vectors [$Y_1, Y_2, Y3, \ldots Y_n$]
- This is known as single head attention.
- For a large and diverse dataset, we may have several parallel contexts.
- We may have to include parallel attention stacks to learn parallel contexts

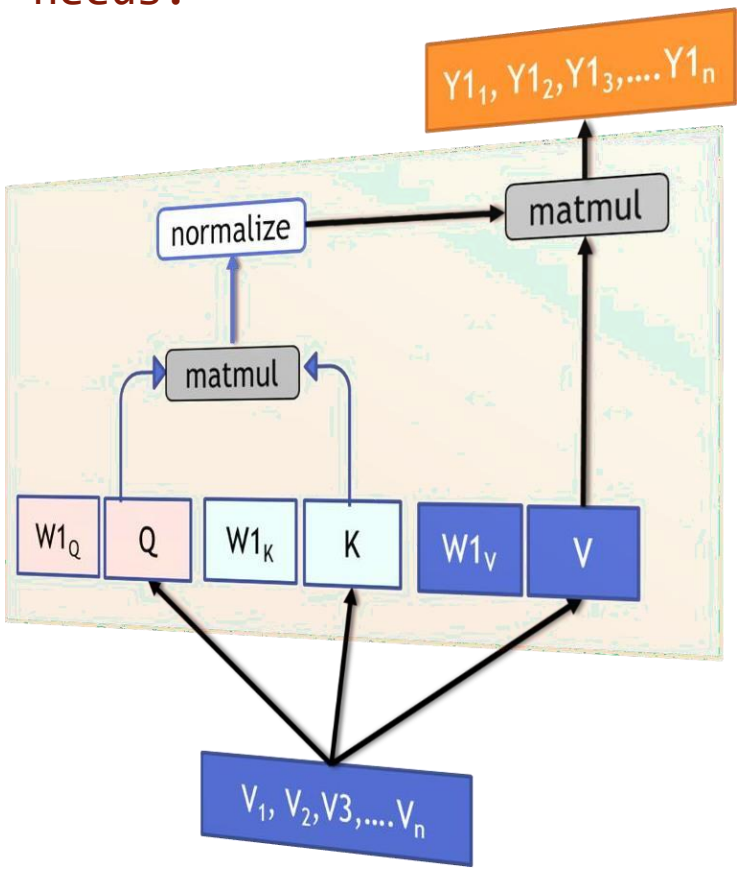
Attention- with two heads



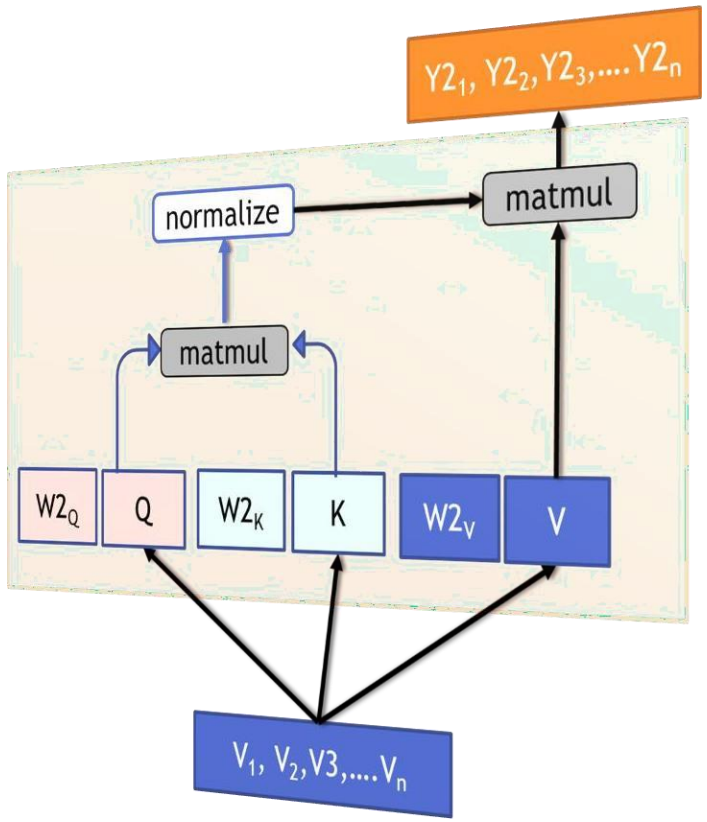


Multi-head Attention.

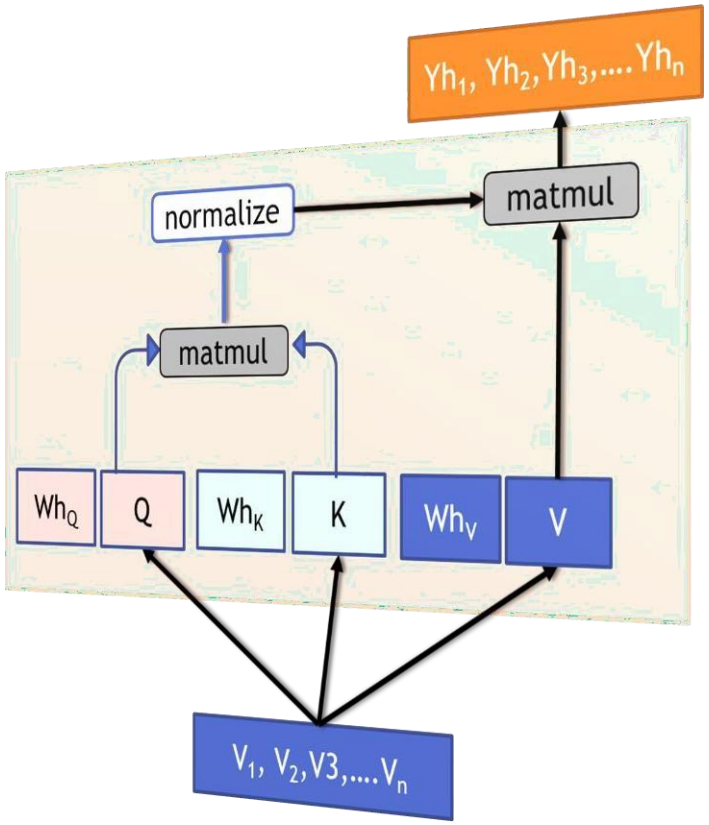
The **bank** offers various **credit cards** for diverse customer needs.



Bank shares experienced a **stock price decline** during this quarter.



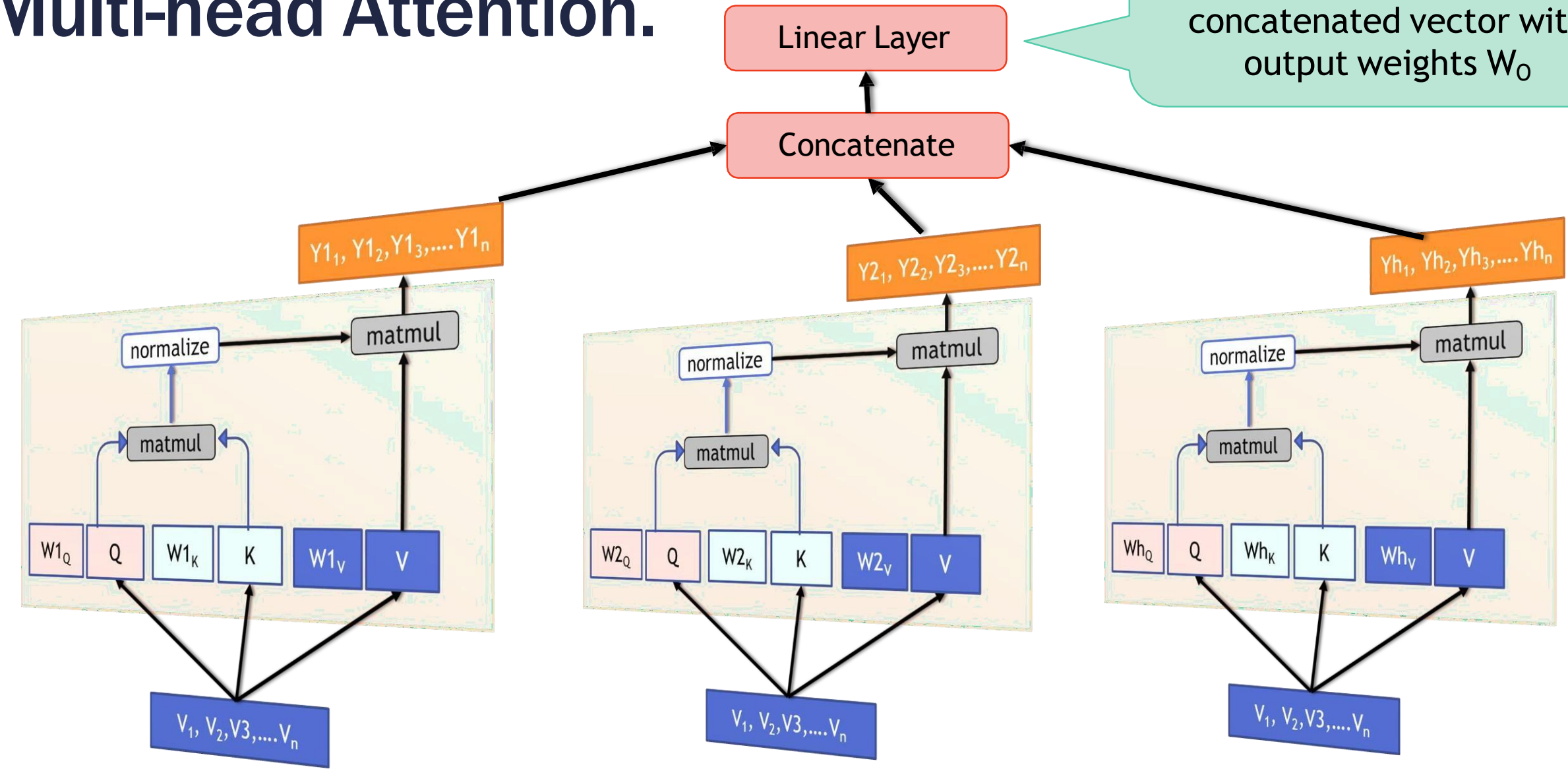
The **bank** sponsors a **football tournament** to expand its customer base.



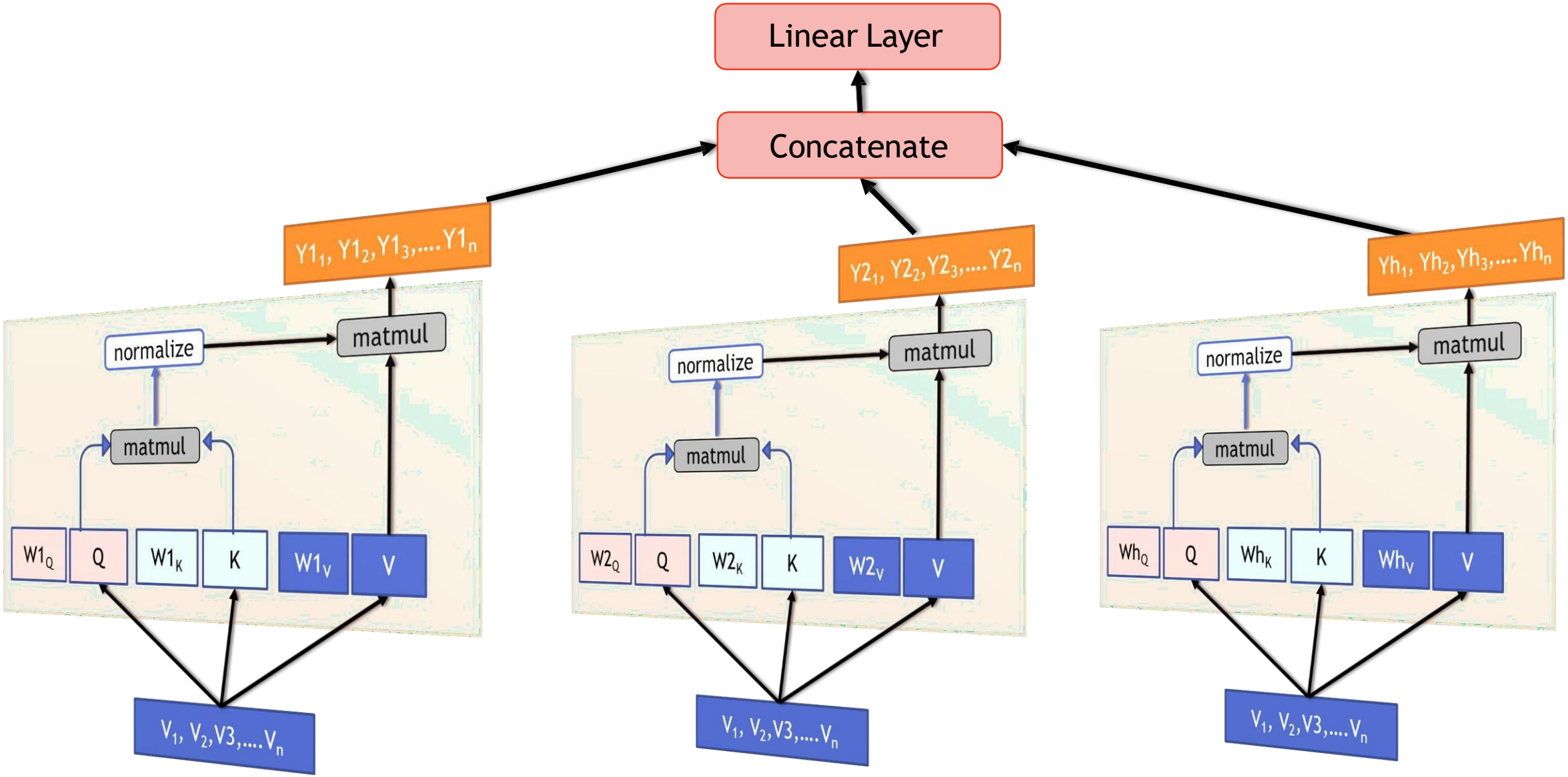
Single-head vs Multi-head Attention.

- Single-head attention can create unique embeddings for the word "bank," depending on its use in phrases like "river bank" or "savings bank."
- Multi-head attention takes this further by assigning the word "bank" different embeddings for its roles in different sub-contexts, even within a larger context like "savings bank."

Multi-head Attention.

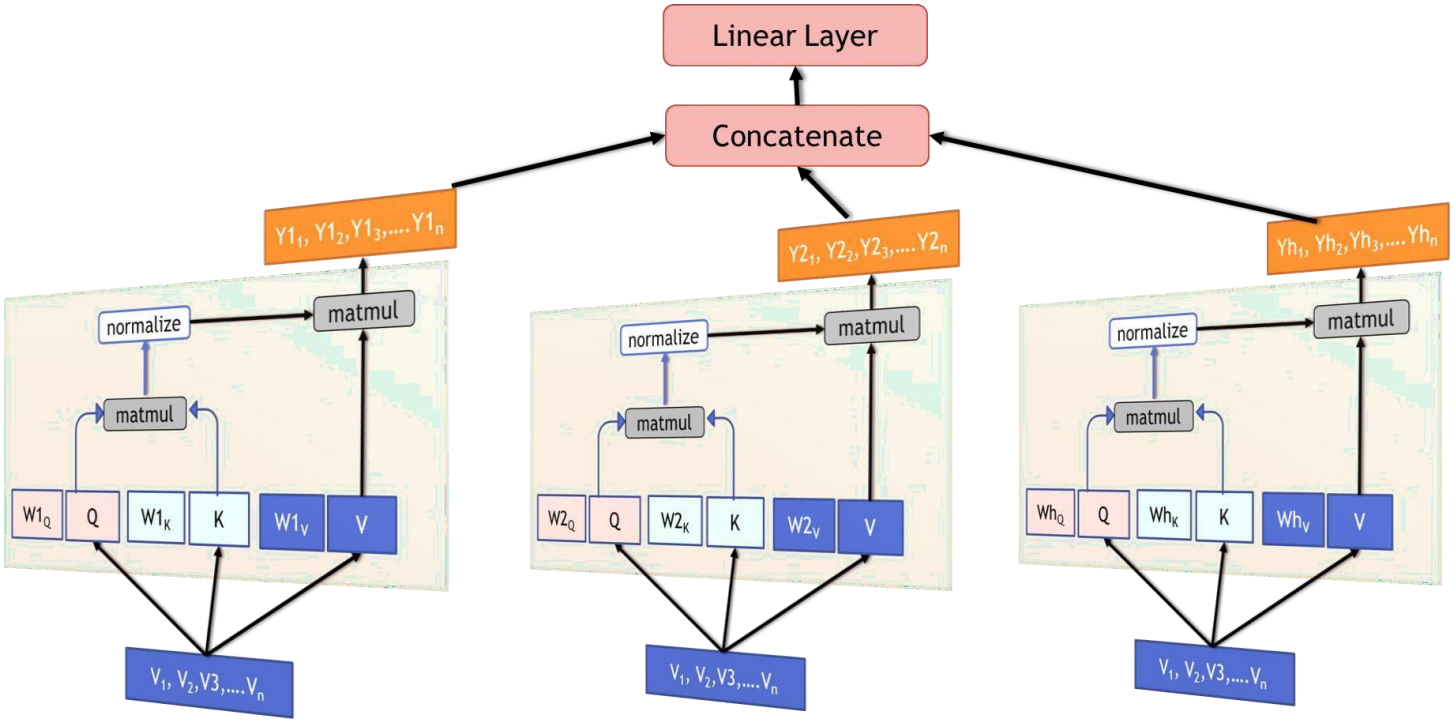


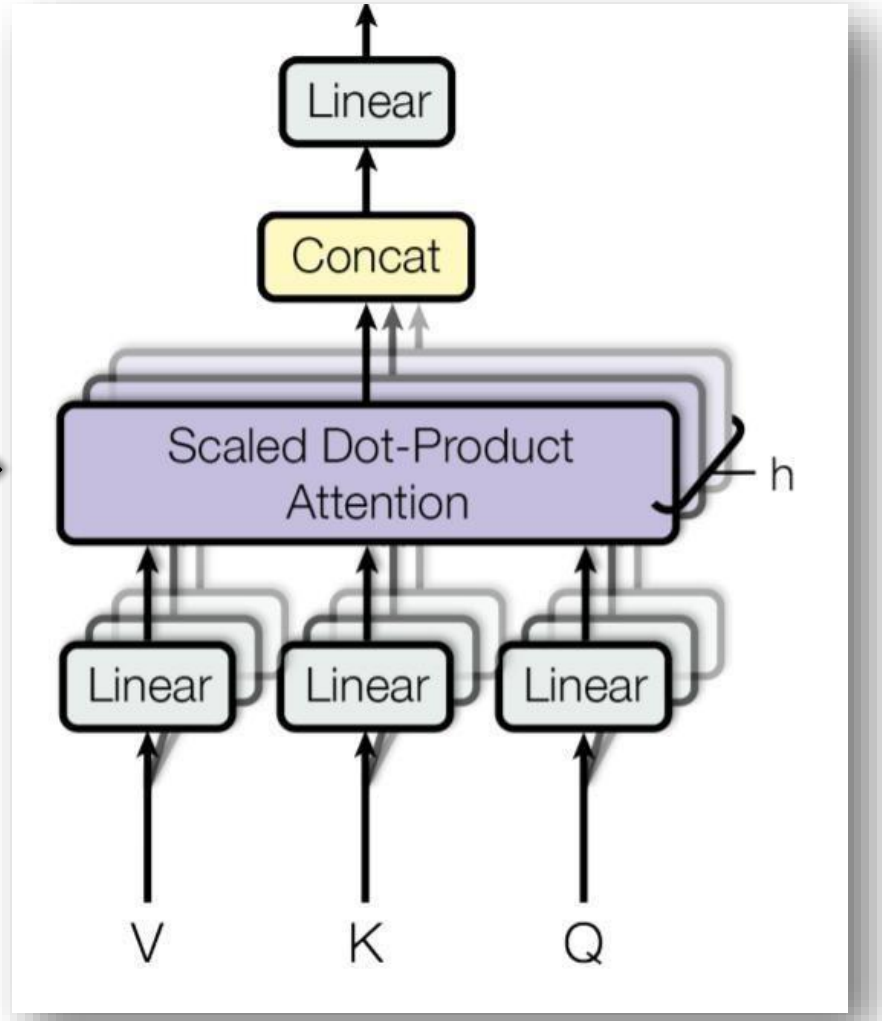
$$MultiHead\ Attention(Q,K,V) = concat(H_1,H_2,..,Hh)\mathrm{W_o}$$
$$where\ Hi = Attention(QW_{iQ},KWik,VWiv)$$



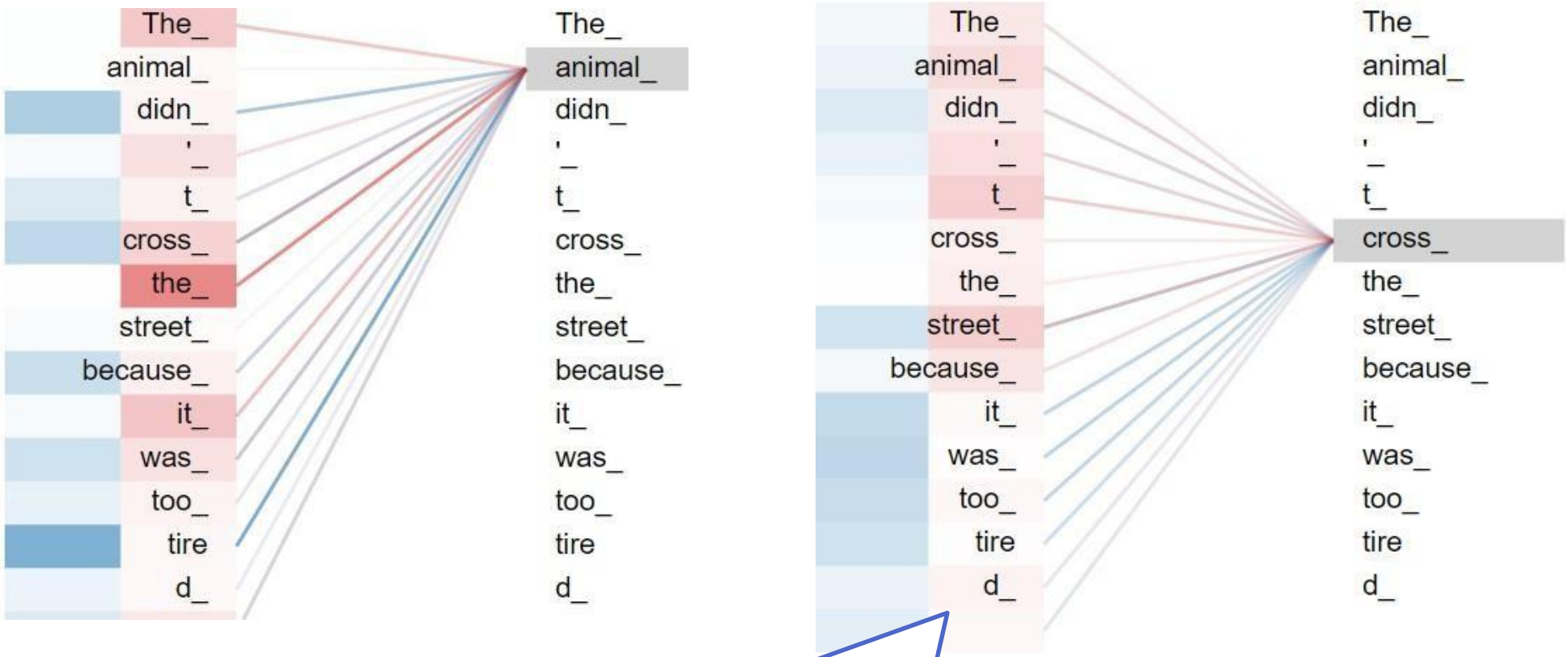
Multi-head Attention –

Actual figure from the paper-Attention is all you need



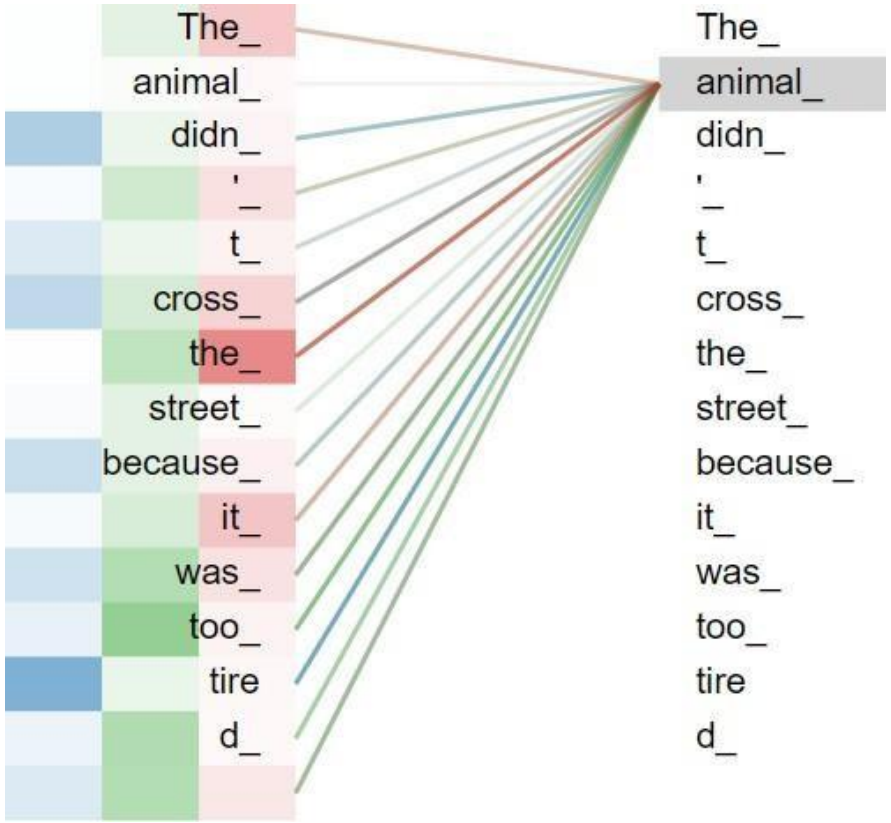


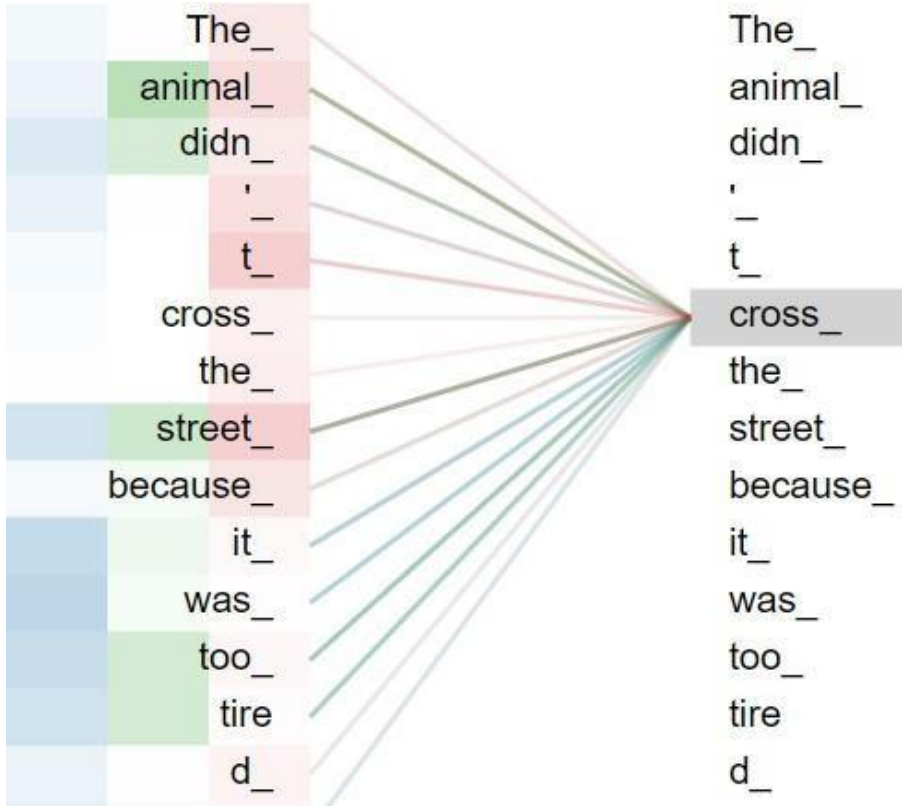
Multi-head Attention visualization - Example



Two attention heads focusing on two different contexts w.r.t a word

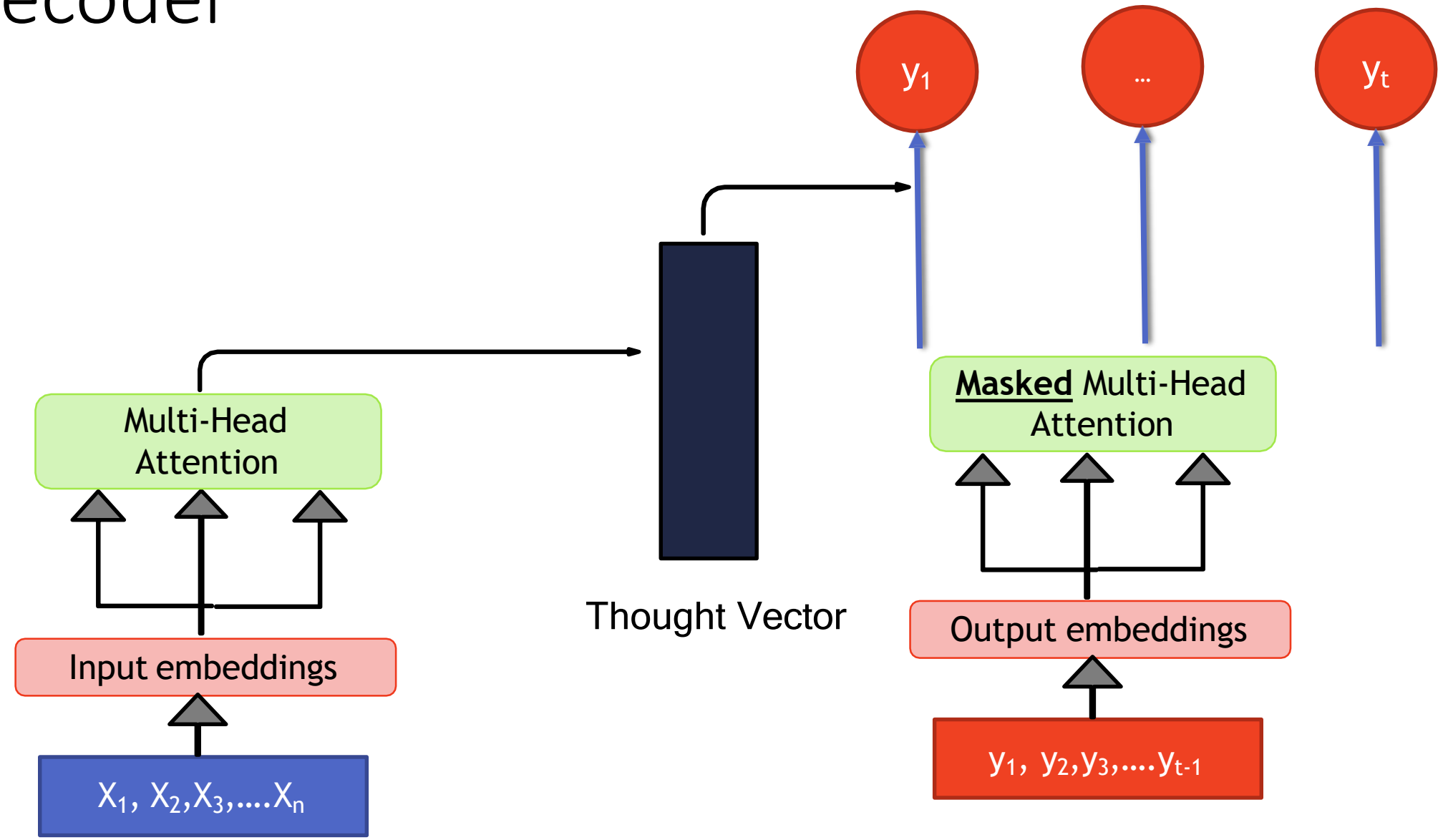
Multi-head Attention visualization - Example





Three attention heads focusing on three different contexts w.r.t the same word

Multi-Head Attention + Encoder and Decoder



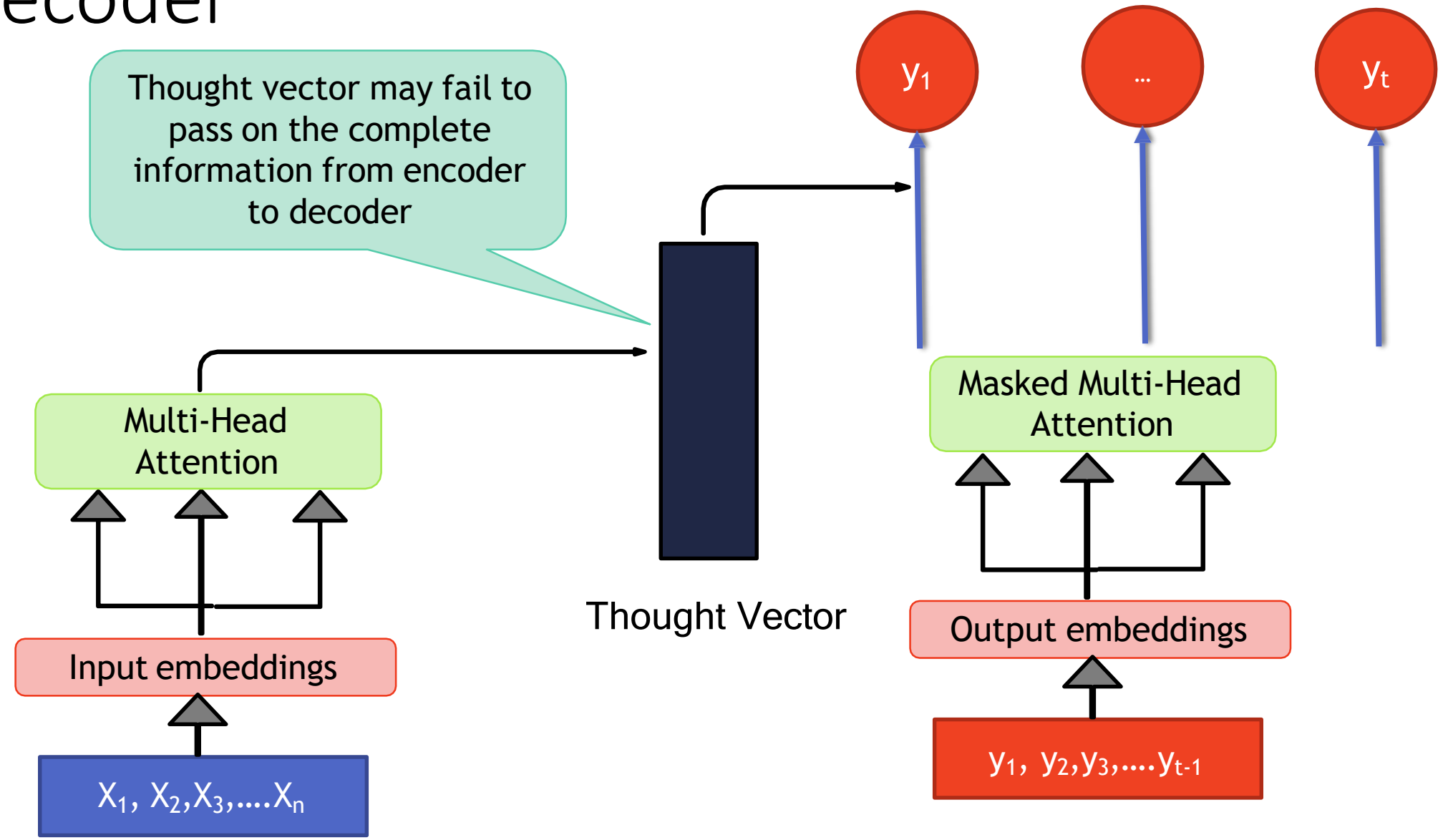
Masked Attention

- Attention in decoder is known as masked attention.
- Why is it masked ?

What is Masked Multi-Head Attention?

English - Encoder	Spanish - Decoder
A nearly perfect photo	**Una foto casi perfecta**
• Inside encoder self attention, while calculating the attention for the word “nearly” • do we consider all the rest of the words?	• Inside decoder self attention, while calculating the attention for the word “casi” • can we consider all the rest of the words?
• We get all the input in one shot.	• We can use only words until now, for the prediction. While predicting word at time-t we can use only words until time t-1 in the decoder
• At the training and test time we have access to all the words.	• At the test time, we will **NOT** have access to future words.
• Encoder self attention doesn’t have any special changes	• Decoder Attention calculation need to be masked from the future words.

Multi-Head Attention + Encoder and Decoder



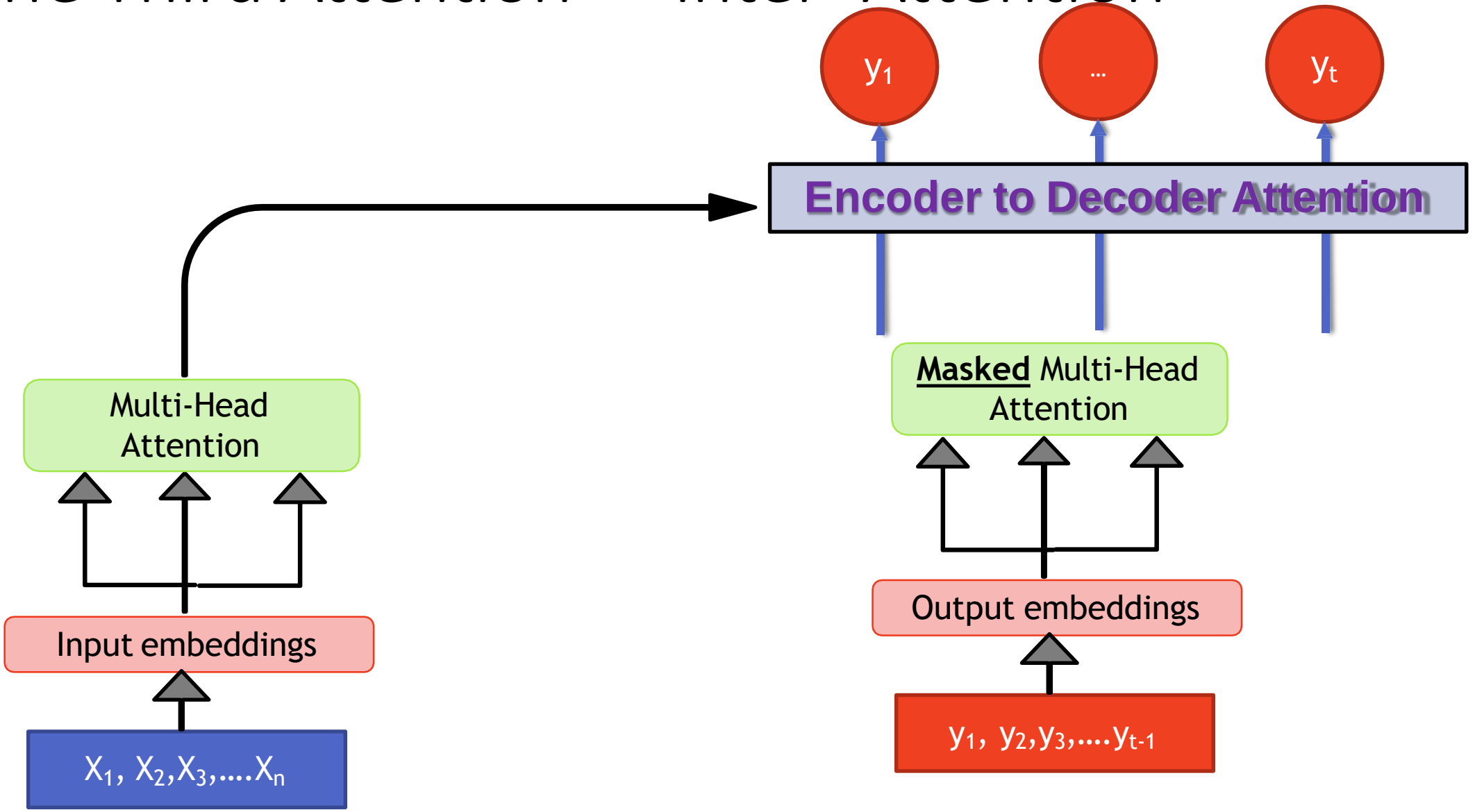
Inter Attention

- The decoder should not only use its own context but also get context from the encoder.
- This can be done by using inter-attention between the encoder and decoder.
- Here, the importance (weights) of each decoder word depends on the words from the encoder.

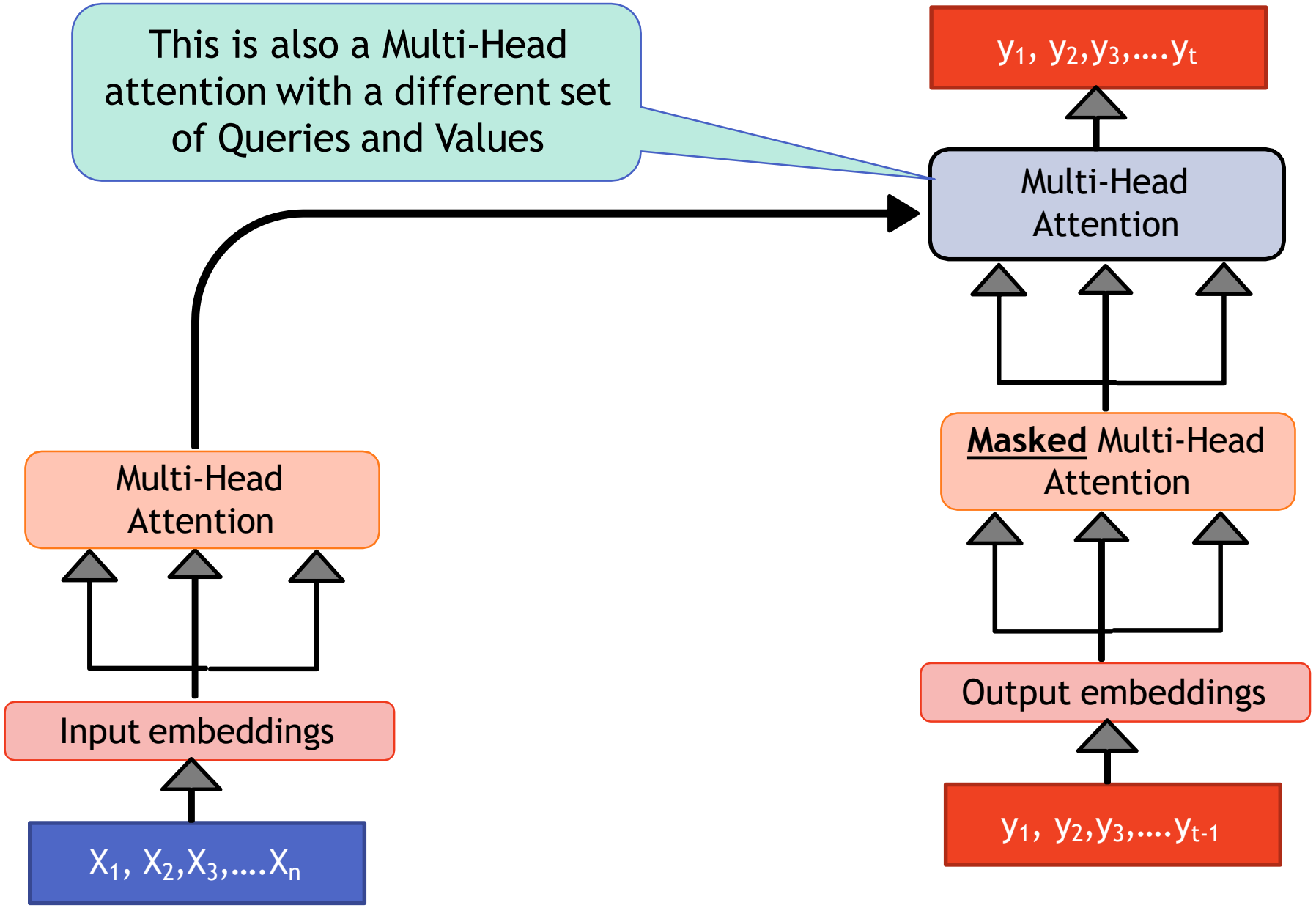
The Third Attention – “Inter- Attention”

- To solve sequence to sequence models, we need to add three different type of attentions.
- For example we are building a machine translation model from English to Spanish.
  - We need an attention mechanism in the input language(English) to understand the contexts in English.
  - We need an attention mechanism in the output language(Spanish) to understand the word relations in Spanish
  - The third attention is the most important one, the word relations and contextual relevance from English to Spanish.

The Third Attention – “Inter- Attention”



Attention + Encoder and Decoder



Q, K, V for Self(Intra) Attention

A	nearly	perfect	photo
-1.1	-1.2	0.1	-2.3
-2.6	-6.5	-0.1	-3.1
-1.7	-3.4	-0.4	-10.5
0.9	7.4	0.0	2.0

(V)

Q, K and V are considered fro

attention block

-4.54
-12.29
-15.90
10.29

(Q)

A	-0.7	0.3	0.2	0.6
nearly	-1.5	1.5	-0.2	0.8
perfect	-1.0	0.8	-0.8	2.8
photo	0.5	-1.7	0.0	-0.5

1.72 -4.43 0.42 -3.77

A	nearly	perfect	photo
-0.7	0.3	0.2	0.6
-1.5	1.5	-0.2	0.8
-1.0	0.8	-0.8	2.8
0.5	-1.7	0.0	-0.5

(K)

Attention / Inter- Attention

- For example, when we are translating from English to Spanish
- Inter attention inside the decoder captures the word relation between decoder and encoder.
  - Q – Queries will be from Spanish (Decoder)
  - K-Keys and V-Values will be from English (Encoder)

Attention / Inter- Attention - Calculation

A	nearly	perfect	photo
-0.1	0.4	0.0	0.0
0.4	-1.4	0.1	0.1
0.2	-1.2	-0.2	0.2
-0.1	1.3	-0.1	0.0

(V)

Una	-0.1	0.0	0.0	0.5
foto	-0.6	0.7	-0.1	0.8
casi	-0.5	0.4	-0.2	0.8
perfecta	0.1	-0.8	0.0	-0.2

0.39
-0.79
-0.90
1.09

(Q)

Queries-Q are from decoder

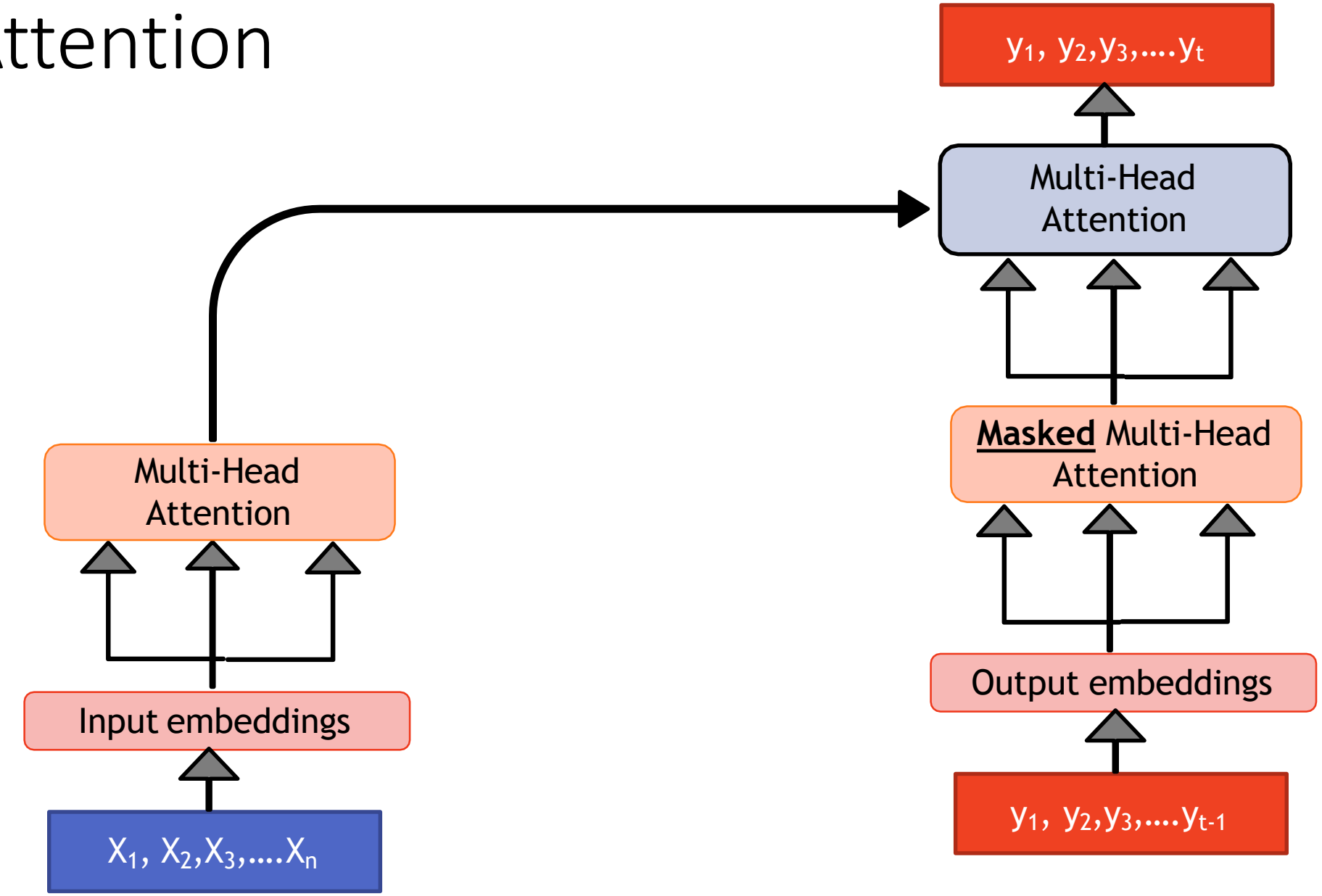
Keys-K and Values-V are from encoder

-0.15	-0.66	0.12	0.07

A	nearly	perfect	photo
0.3	-0.6	0.1	0.2
-2.3	2.1	0.8	1.6
-1.6	1.8	-1.3	2.4
0.9	-2.0	-0.5	-0.1

(K)

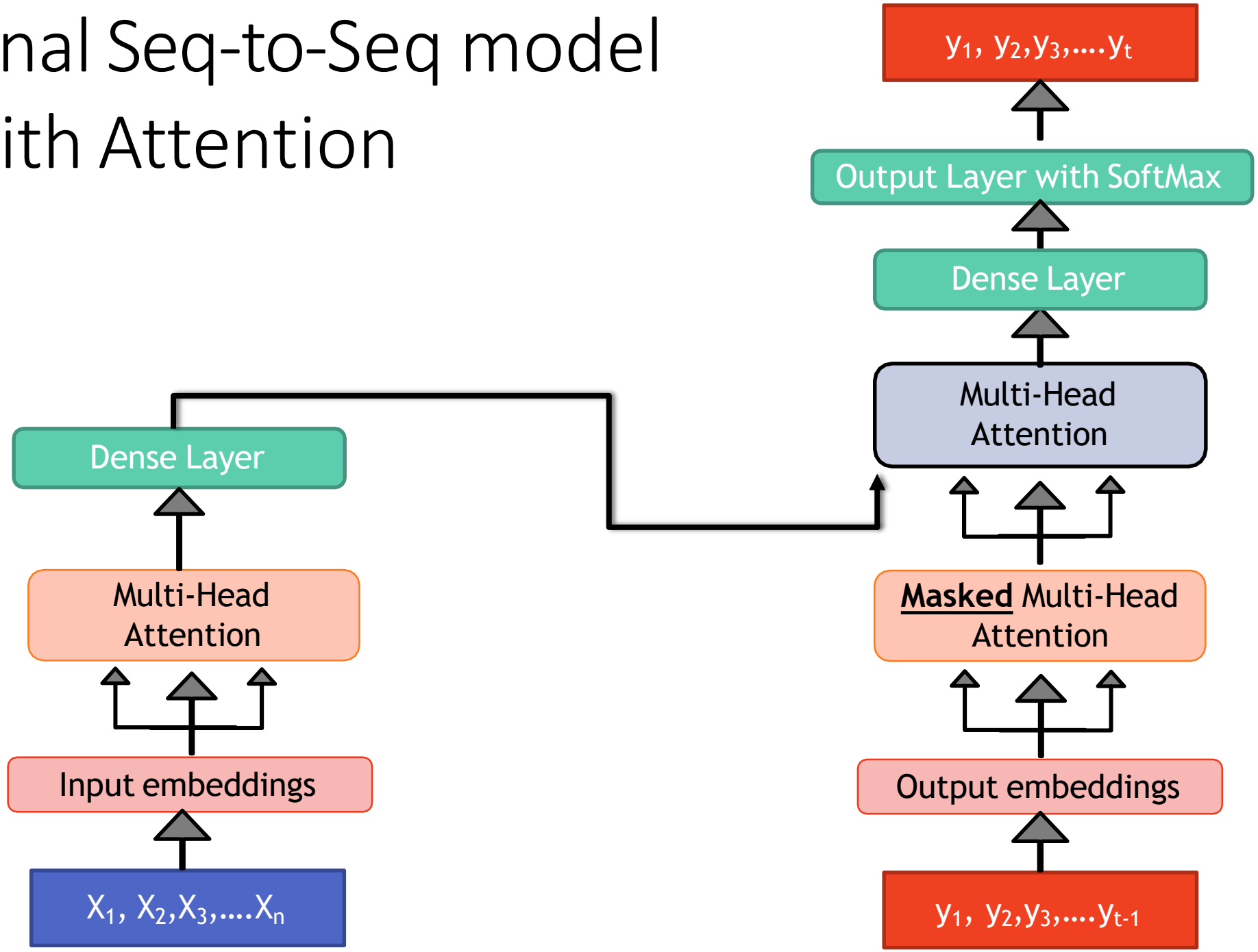
Seq-to-Seq model with Attention



Attention + Dense Layers for learning

- Until now, we have included only attention blocks.
- But eventually, these attention blocks will give us just better embeddings only.
- We need to add at least a couple of **dense layers** on top of these intelligent embeddings.
- These dense layers will help us in learning all the patterns in the data.
- Dense layers are also known as hidden layers or feed forward layers

Final Seq-to-Seq model with Attention



Preserving the sequential order

- Questions: In word2vec do we preserve the sequential order? In attention are we preserving the sequential order?
- Answer: Nope, we are not. Context is decided based on the surrounding words, not necessarily by retaining the sequence.

1. “the food was good, not bad”
2. “the food was bad, not good”

Self attention treats both the sentences the same way. The order and the position of “good” and “bad” are very important in NLP.

Positional encodings

- How do we include the information about the sequential order or the position of the sentence?
- By using “Positional encodings”
- It is a very simple concept. We will create a new vector based on the relative position of the word in a sentence.
- We will now add the positional information to the embeddings and pass it on as input

Positional encodings

- In models like **Transformers**, there's **no inherent notion of sequence order** (unlike RNNs or CNNs).
- To let the model understand the **order of elements in a sequence** (like words in a sentence), we use **positional encodings**.
- Transformers process all tokens **in parallel**—they don't know which word comes first or last. So, we inject **positional information** into the input embedding's.

- **Sinusoidal Positional Encoding (Static)**
  Introduced in the original **Vaswani et al. (2017)** Transformer paper. It uses sine and cosine functions of different frequencies. No additional parameters are learned and purely mathematical.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Simple Positional encodings

- In the first sentence, good has value-3 and bad has value-5
- In the second sentence, good has value-5 and bad has value-3

the	food	was	good	not	bad
0	1	2	3	4	5

the	food	was	bad	not	good
0	1	2	3	4	5

- We can add this extra information to input embeddings and pass it on as the input for attention calculation

Adding Positional encodings - idea

the	food	was	good	not	bad
-0.2	-0.2	-1.1	3.2	-0.2	-0.3
-1.7	0.9	-0.9	3.7	-0.5	1.0
-0.2	-0.5	0.3	2.6	0.3	-2.2
-1.3	-0.8	0.1	2.5	2.6	-1.5



the	
-0.2	
-1.7	
-0.2	
-1.3	

food	
-0.2	
0.9	
-0.5	
-0.8	

was	
-1.1	
-0.9	
0.3	
0.1	

good	
3.2	
3.7	
2.6	
2.5	

not	
-0.2	
-0.5	
0.3	
2.6	

bad	
-0.30	
1.00	
-2.20	
-1.50	

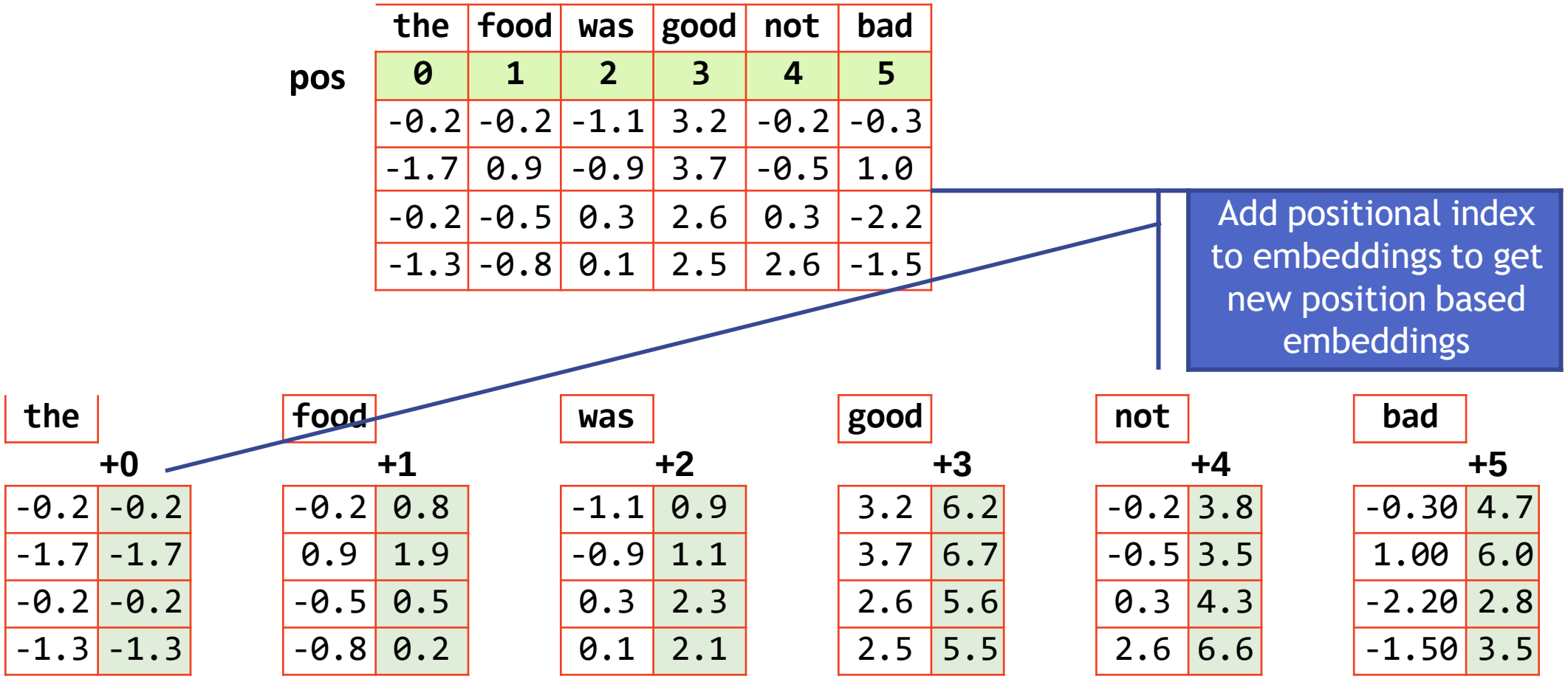
Adding Positional encodings - idea

pos	the	food	was	good	not	bad
	0	1	2	3	4	5
	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
	-1.7	0.9	-0.9	3.7	-0.5	1.0
	-0.2	-0.5	0.3	2.6	0.3	-2.2
	-1.3	-0.8	0.1	2.5	2.6	-1.5

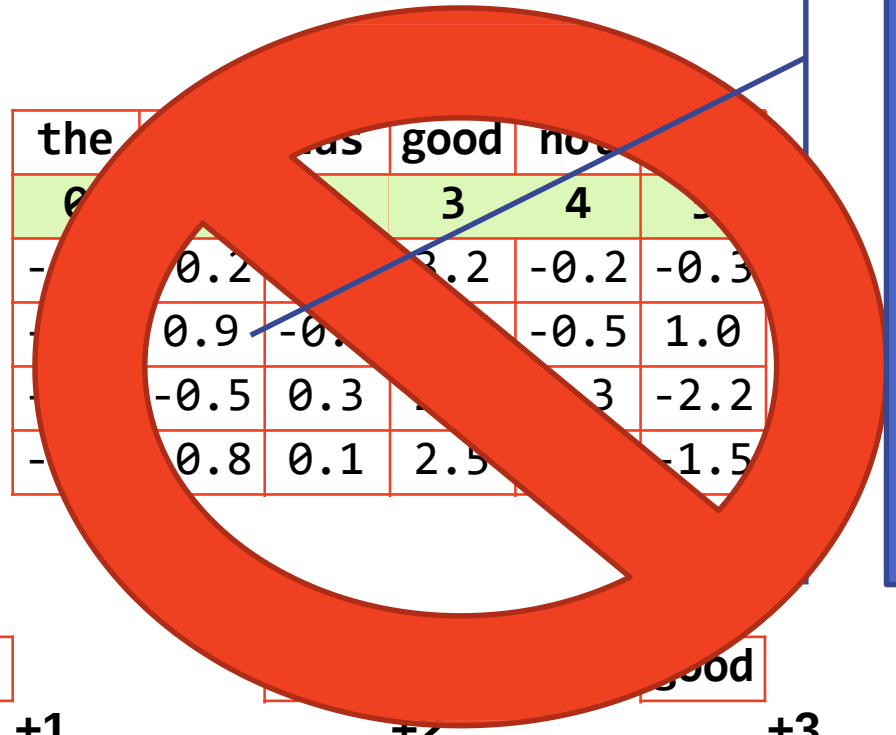
Positional index

the		food		was		good		not		bad	
-0.2		-0.2		-1.1		3.2		-0.2		-0.30	
-1.7		0.9		-0.9		3.7		-0.5		1.00	
-0.2		-0.5		0.3		2.6		0.3		-2.20	
-1.3		-0.8		0.1		2.5		2.6		-1.50	

Adding Positional encodings - idea



Adding Positional encodings - idea



- This method doesn't work.
- If the number of words are too many inside a sequence then we will have very large values.
- If there are 50 words in a sentence, then we will be adding 50 to the final word.
- We need a slightly modified function for positional encodings.

the	
+0	
-0.2	-0.2
-1.7	-1.7
-0.2	-0.2
-1.3	-1.3

food	
+1	
-0.2	0.8
0.9	1.9
-0.5	0.5
-0.8	0.2

+2	
-1.1	0.9
-0.9	1.1
0.3	2.3
0.1	2.1

+3	
3.2	6.2
3.7	6.7
2.6	5.6
2.5	5.5

not	
+4	
-0.2	3.8
-0.5	3.5
0.3	4.3
2.6	6.6

bad	
+5	
-0.30	4.7
1.00	6.0
-2.20	2.8
-1.50	3.5

New formula Positional encodings

pos	the	food	was	good	not	bad
	0	1	2	3	4	5
	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
	-1.7	0.9	-0.9	3.7	-0.5	1.0
	-0.2	-0.5	0.3	2.6	0.3	-2.2
	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$
$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

the	
-0.2	?
-1.7	?
-0.2	?
-1.3	?

food	
-0.2	?
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

New formula Positional encodings

pos	the	food	was	good	not	bad
	0	1	2	3	4	5
	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
	-1.7	0.9	-0.9	3.7	-0.5	1.0
	-0.2	-0.5	0.3	2.6	0.3	-2.2
	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$
$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

the	
-0.2	?
-1.7	?
-0.2	?
-1.3	?

food	
-0.2	?
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

New formula Positional encodings

	the	food	was	good	not	bad
pos	0	1	2	3	4	5
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
i=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
i=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
i=3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the	
-0.2	?
-1.7	?
-0.2	?
-1.3	?

food	
-0.2	?
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

Positional encodings - Calculation

$pos = 0\ ;\ i = 0\ ;\ d = 4$

$$PE(0,0) = \sin\left(\frac{0}{10000^{\frac{2*0}{4}}}\right)$$

	the	food	was	good	not	bad
pos	**0**	**1**	**2**	**3**	**4**	**5**
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
i=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
i=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
i=3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the	
-0.2	0
-1.7	
-0.2	
-1.3	

food	
-0.2	?
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

Positional encodings - Calculation

$pos = 0\ ;\ i = 1\ ;\ d = 4$

$$PE(0,1) = \sin\left(\frac{0}{10000^{\frac{2*0.5}{4}}}\right)$$

	the	food	was	good	not	bad
pos	**0**	**1**	**2**	**3**	**4**	**5**
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
i=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
i=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
i=3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the	
-0.2	0
-1.7	0
-0.2	
-1.3	

food	
-0.2	?
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

Positional encodings - Calculation

$pos = 1\ ;\ i = 0\ ;\ d = 4$

$$PE(1,0) = \sin\left(\frac{1}{10000^{\frac{0}{4}}}\right)$$

the	food	was	good	not	bad
0	1	2	3	4	5
-0.2	-0.2	-1.1	3.2	-0.2	-0.3
-1.7	0.9	-0.9	3.7	-0.5	1.0
-0.2	-0.5	0.3	2.6	0.3	-2.2
-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the	
-0.2	0
-1.7	0
-0.2	0
-1.3	0

food	
-0.2	0.84
0.9	?
-0.5	?
-0.8	?

was	
-1.1	?
-0.9	?
0.3	?
0.1	?

good	
3.2	?
3.7	?
2.6	?
2.5	?

not	
-0.2	?
-0.5	?
0.3	?
2.6	?

bad	
-0.30	?
1.00	?
-2.20	?
-1.50	?

Positional encodings - Calculation

$pos = 1\ ;\ i = 1\ ;\ d = 4$

$$PE(1,1) = \sin\left(\frac{1}{10000^{\frac{1}{4}}}\right)$$

	the	food	was	good	not	bad
pos	0	1	2	3	4	5
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the		foo		was		good		not		bad	
-0.2	0	-0.2	0.84	-1.1	?	3.2	?	-0.2	?	-0.30	?
-1.7	0	0.9	0.10	-0.9	?	3.7	?	-0.5	?	1.00	?
-0.2	0	-0.5	?	0.3	?	2.6	?	0.3	?	-2.20	?
-1.3	0	-0.8	?	0.1	?	2.5	?	2.6	?	-1.50	?

Positional encodings - Calculation

$pos = 1\ ;\ i = 2\ ;\ d = 4$

$$PE(1,2) = \sin\left(\frac{1}{10000^{\frac{2}{4}}}\right)$$

	the	food	was	good	not	bad
pos	0	1	2	3	4	5
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the		fo		was		good		not		bad	
-0.2	0	-0.2	0.84	-1.1	?	3.2	?	-0.2	?	-0.30	?
-1.7	0	0.9	0.10	-0.9	?	3.7	?	-0.5	?	1.00	?
-0.2	0	-0.5	0.01	0.3	?	2.6	?	0.3	?	-2.20	?
-1.3	0	-0.8		0.1	?	2.5	?	2.6	?	-1.50	?

Positional encodings - Calculation

$pos = 1\ ;\ i = 2\ ;\ d = 4$

$$PE(1,2) = \sin\left(\frac{1}{10000^{\frac{2}{4}}}\right)$$

	the	food	was	good	not	bad
pos	0	1	2	3	4	5
i=0	-0.2	-0.2	-1.1	3.2	-0.2	-0.3
=1	-1.7	0.9	-0.9	3.7	-0.5	1.0
=2	-0.2	-0.5	0.3	2.6	0.3	-2.2
3	-1.3	-0.8	0.1	2.5	2.6	-1.5

$$Positional\ Encoding$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

In our example

- d - is the dimensionality of the embeddings ; d=4 here
- i - runs from 0 to 3
- pos - Takes values 0 to 5

the		fo		was		good		not		bad	
-0.2	0	-0.2	0.84	-1.1	?	3.2	?	-0.2	?	-0.30	?
-1.7	0	0.9	0.10	-0.9	?	3.7	?	-0.5	?	1.00	?
-0.2	0	-0.5	0.01	0.3	?	2.6	?	0.3	?	-2.20	?
-1.3	0	-0.8		0.1	?	2.5	?	2.6	?	-1.50	?

Positional encodings - Calculation

the	food	was	good	not	bad
-0.2	-0.2	-1.1	3.2	-0.2	-0.3
-1.7	0.9	-0.9	3.7	-0.5	1.0
-0.2	-0.5	0.3	2.6	0.3	-2.2
-1.3	-0.8	0.1	2.5	2.6	-1.5

Positional encodings for all the words in sentence-1

the	
-0.2	0
-1.7	0
-0.2	0
-1.3	0

food	
-0.2	0.84
0.9	0.100
-0.5	0.010
-0.8	0.001

was	
-1.1	0.909
-0.9	0.199
0.3	0.020
0.1	0.002

good	
3.2	0.141
3.7	0.296
2.6	0.030
2.5	0.003

not	
-0.2	-0.288
-0.5	0.002
0.3	0.000
2.6	0.000

bad	
-0.30	0.913
1.00	0.002
-2.20	0.000
-1.50	0.000

Embeddings + Positional encodings

the	food	was	good	not	bad
0	1	2	3	4	5
-0.2	-0.2	-1.1	3.2	-0.2	-0.3
-1.7	0.9	-0.9	3.7	-0.5	1.0
-0.2	-0.5	0.3	2.6	0.3	-2.2
-1.3	-0.8	0.1	2.5	2.6	-1.5

Embeddings updated with positional encoders

the	food	was	good	not	bad
0	1	2	3	4	5
-0.2	0.6	-0.2	3.3	-0.5	0.6
-1.7	1.0	-0.7	4.0	-0.5	1.0
-0.2	-0.5	0.3	2.6	0.3	-2.2
-1.3	-0.8	0.1	2.5	2.6	-1.5

the	
-0.2	0
-1.7	0
-0.2	0
-1.3	0

food	
-0.2	0.84
0.9	0.100
-0.5	0.010
-0.8	0.001

was	
-1.1	0.909
-0.9	0.199
0.3	0.020
0.1	0.002

good	
3.2	0.141
3.7	0.296
2.6	0.030
2.5	0.003

not	
-0.2	-0.288
-0.5	0.002
0.3	0.000
2.6	0.000

bad	
-0.30	0.913
1.00	0.002
-2.20	0.000
-1.50	0.000

Sentence-2

the	food	was	bad	not	good
0	1	2	3	4	5
-0.2	-0.2	-1.1	-0.3	-0.2	3.2
-1.7	0.9	-0.9	1.0	-0.5	3.7
-0.2	-0.5	0.3	-2.2	0.3	2.6
-1.3	-0.8	0.1	-1.5	2.6	2.5

the	food	was	bad	not	good
0	1	2	3	4	5
-0.2	0.6	-0.2	-0.2	-0.5	4.1
-1.7	1.0	-0.7	1.3	-0.5	3.7
-0.2	-0.5	0.3	-2.2	0.3	2.6
-1.3	-0.8	0.1	-1.5	2.6	2.5

the	
-0.2	0.0
-1.7	0.0
-0.2	0.0
-1.3	0.0

food	
-0.2	0.84
0.9	0.100
-0.5	0.010
-0.8	0.001

was	
-1.1	0.909
-0.9	0.199
0.3	0.020
0.1	0.002

bad	
-0.3	0.141
1.0	0.296
-2.2	0.030
-1.5	0.003

not	
-0.2	-0.288
-0.5	0.002
0.3	0.000
2.6	0.000

good	
3.2	0.913
3.7	0.002
2.6	0.000
2.5	0.000

Sentence-1 vs Sentence-2 with positional **encodings**

Have you noticed the difference?

Sentence-1					
the	food	was	good	not	bad
0	1	2	3	4	5
-0.2	0.6	-0.2	3.3	-0.5	0.6
-1.9	1.0	-0.7	4.0	-0.5	1.0
-1.2	-0.5	0.3	2.6	0.3	-2.2
-1.5	-0.8	0.1	2.5	2.6	-1.5

Sentence-2					
the	food	was	bad	not	good
0	1	2	3	4	5
-0.2	0.6	-0.2	-0.2	-0.5	4.1
-1.9	1.0	-0.7	1.3	-0.5	3.7
-1.2	-0.5	0.3	-2.2	0.3	2.6
-1.5	-0.8	0.1	-1.5	2.6	2.5

Sentence-1 vs Sentence-2 with positional encodings

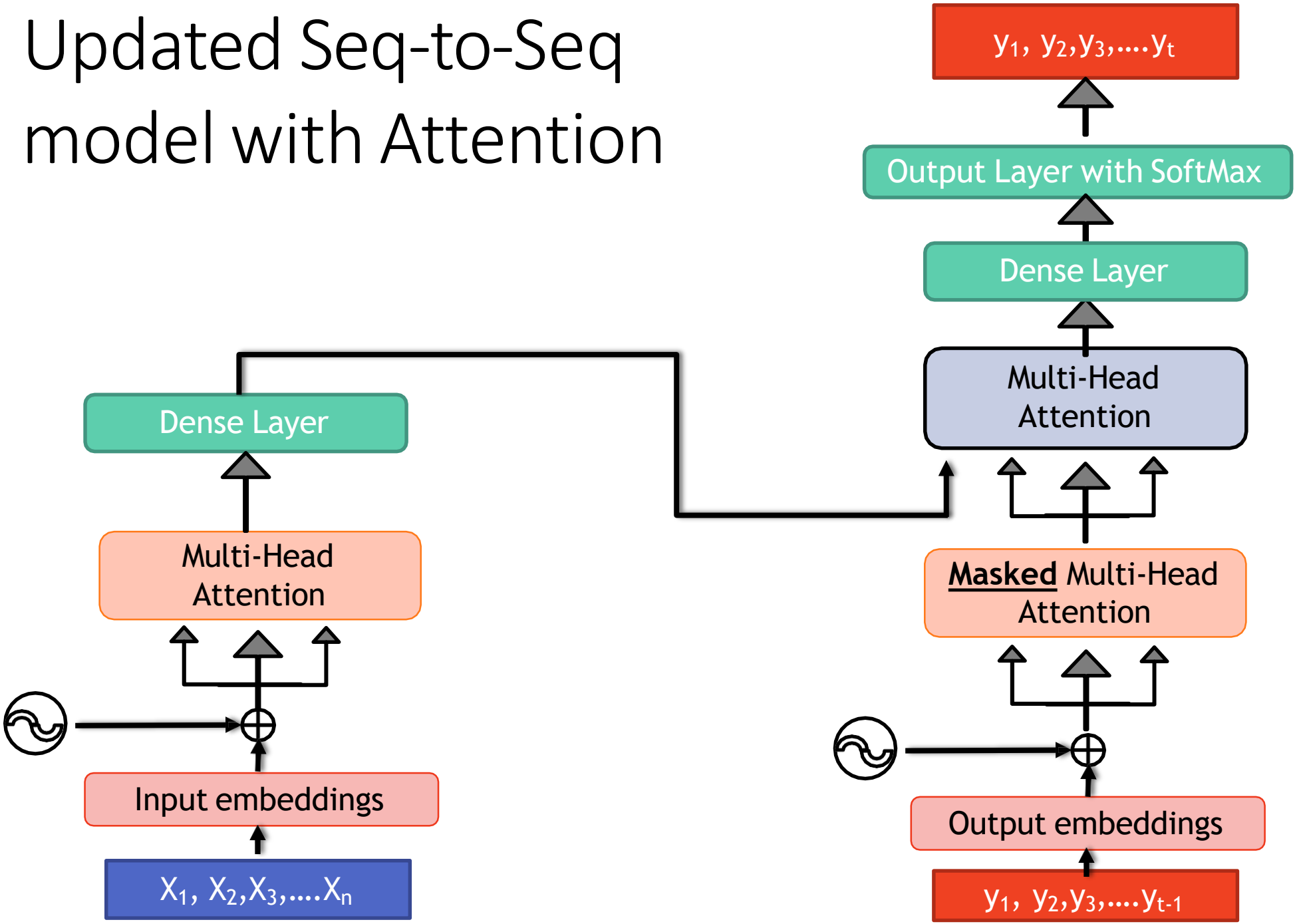
Sentence-1					
the	food	was	good	not	bad
0	1	2	3	4	5
-0.2	0.6	-0.2	3.3	-0.5	0.6
-1.9	1.0	-0.7	4.0	-0.5	1.0
-1.2	-0.5	0.3	2.6	0.3	-2.2
-1.5	-0.8	0.1	2.5	2.6	-1.5

Sentence-2					
the	food	was	bad	not	good
0	1	2	3	4	5
-0.2	0.6	-0.2	-0.2	-0.5	4.1
-1.9	1.0	-0.7	1.3	-0.5	3.7
-1.2	-0.5	0.3	-2.2	0.3	2.6
-1.5	-0.8	0.1	-1.5	2.6	2.5

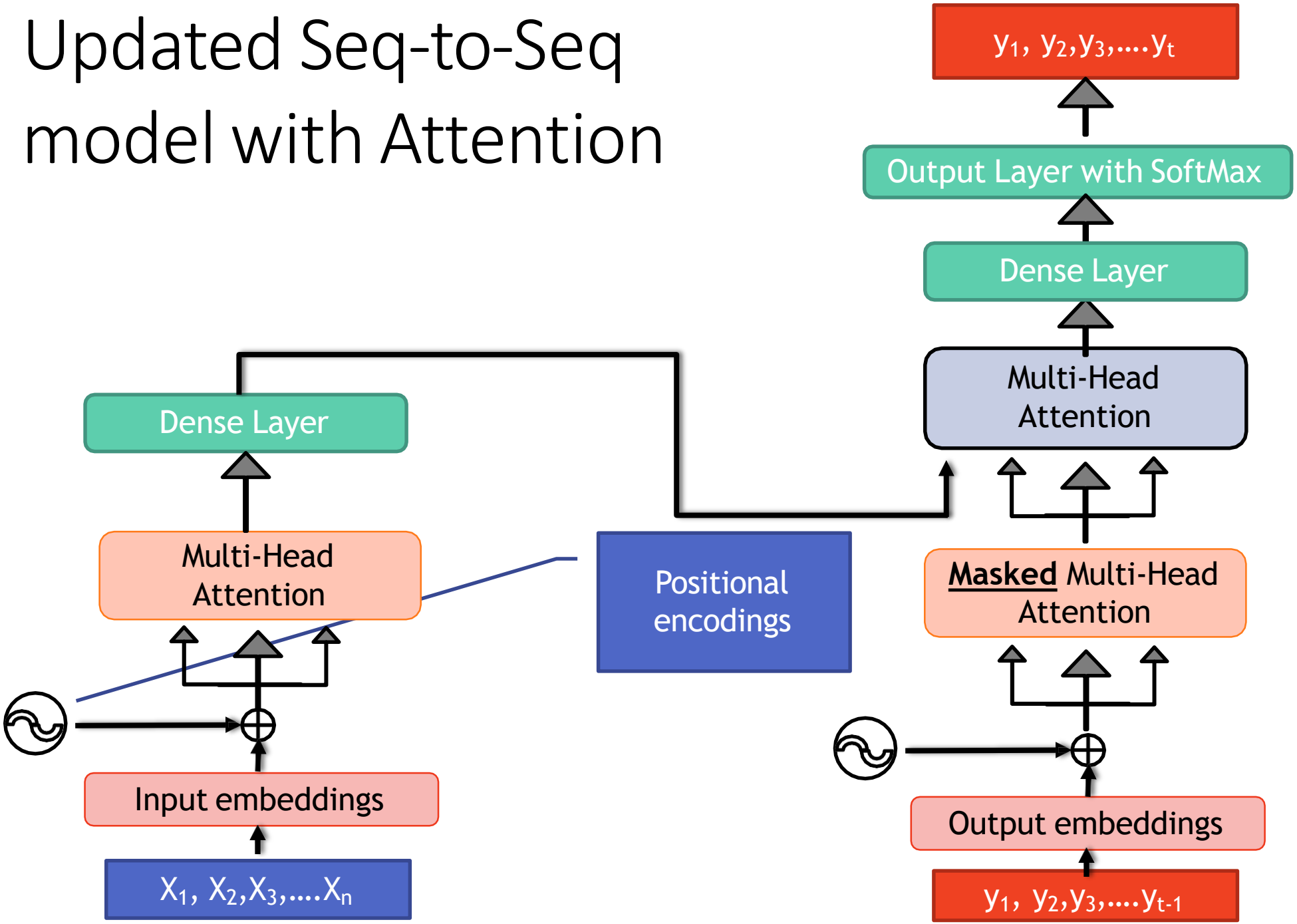
Positional encodings - Conclusion

- Word2vec → numerical representation of the words
- Positional encodings → a numerical representation of the position of the word in the original sentence.
- Positional encodings – An extra vector to capture the position
- Both Word2Vec and Positional encodings are vectors.
- These two vectors are added and passed on to encoder

Updated Seq-to-Seq model with Attention



Updated Seq-to-Seq model with Attention

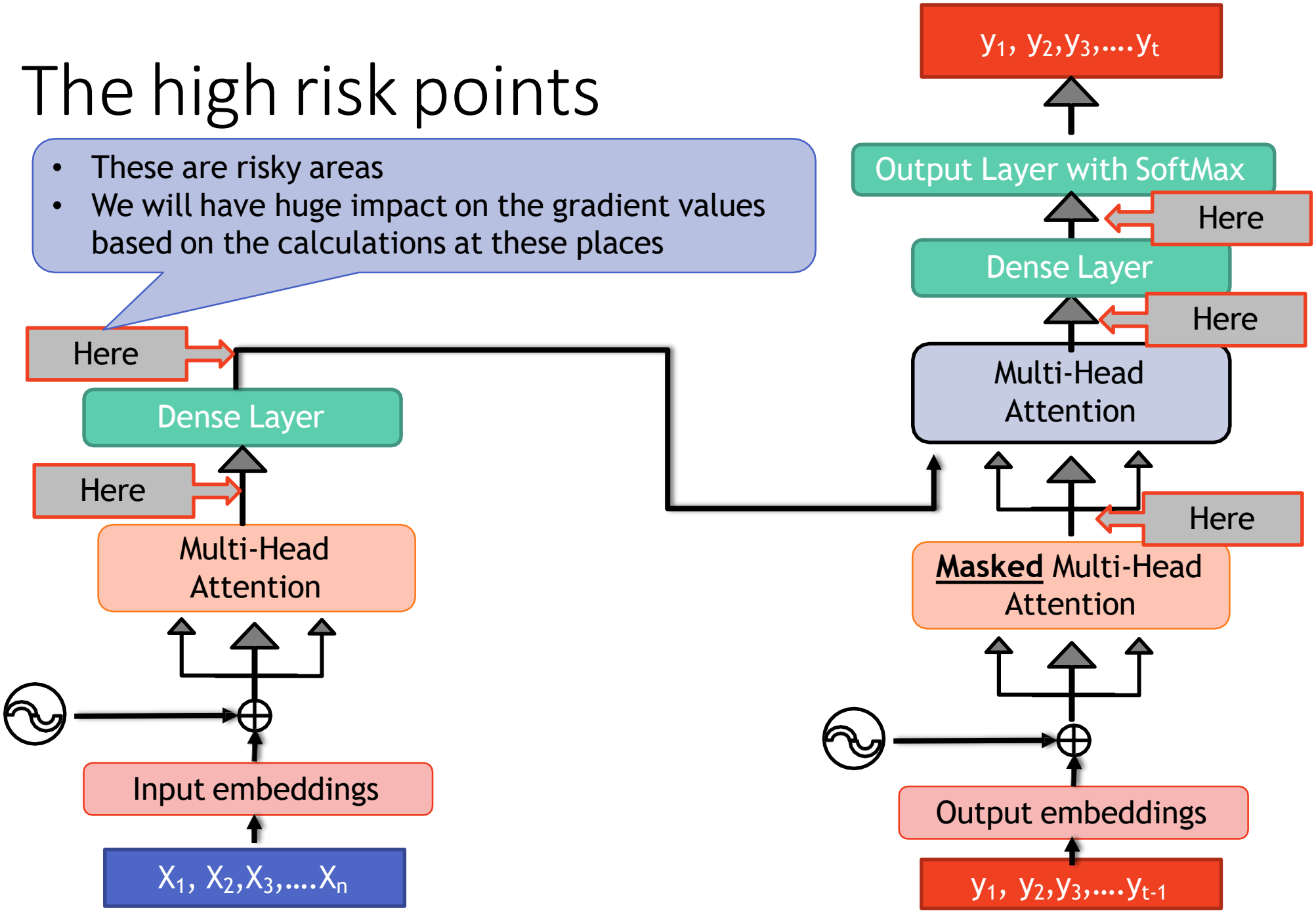


Few more practical issues – The high risk points in the network

- While solving practical problems, DL models would like to have the weights near to zero, both positive and negative.
- If not, we have multiple problems like unstable training, vanishing gradients, etc.,
- There are some layers in the that network that may cause significant impact on the gradient calculation.
- Lets call them the “high risk points”

The high risk points

- These are risky areas
- We will have huge impact on the gradient values based on the calculations at these places



Fixing the practical issues – Two steps

- We will now add two extra elements to our model seq-to-seq model architecture to keep the gradient flow smooth and to make the model mathematically convenient to solve large data problems.
  1) **Residual connection**
  2) **Normalization**
- It's a common practice to use them in any complex model.

Encoder

- **Input:** Token embeddings (with positional encoding).Consists of **N stacked layers** . Each layer contains:
  - **Multi-Head Self-Attention**: Each token attends to all others, learning dependencies.
  - **Feedforward Network (FFN)**: Adds non-linearity and transforms embeddings.
  - **Residual Connections + Layer Normalization**: Preserve information and stabilize training.
- **Output:** **Contextual embeddings** for each input token (a rich representation of the source sequence).

Feedforward Network (FFN)

Input (per token) —▶ Linear Layer —▶ ReLU —▶ Linear Layer —▶ Output (same token)

- It applies non-linear transformations to each position (token) independently.
- It increases the model's capacity to capture complex relationships beyond linear attention mechanisms.

The **FFN** is a simple **two-layer fully connected feed forward network** with a **ReLU** activation in between.

FFN(x)=max(0,XW1+b1)W2+b2

W1 and W2 are the weight matrices for the first and second linear layers, respectively.
b1 and b2 are the biases for these layers.
The ReLU activation is applied element-wise after the first linear transformation.

The output of the FFN provides a refined representation of each input position, which combines the contextual embeddings adjusted for potential non-linear relationships within the data.

Residual Connections

A **residual connection** adds the **input of a layer** directly to its **output**.

Output=Layer(x)+x

Each sub-layer (self-attention, feedforward network) in the Transformer is wrapped with a residual connection:

For Self-Attention:

$$\text{SA_Output} = \text{LayerNorm}(\text{SelfAttention}(x) + x)$$

For Feedforward Network:

$$\text{FFN_Output} = \text{LayerNorm}(\text{FFN}(x) + x)$$

- **Residual connections carry over the previous embeddings to the subsequent layers.**

- **As such, the encoder blocks enrich the embedding vectors with additional information obtained from the multi-head self-attention calculations and position-wise feed-forward networks.**

Why are residual connections important?

Prevent Vanishing Gradient Problem: In deep networks, gradients can shrink as they backpropagate, causing learning to stall. Residual connections ensure gradient flow is preserved by providing a "shortcut" through the network.

Preserve Information: Even if intermediate layers distort the input, the original input x is still passed forward.

Enable Deep Architectures: Transformers use multiple stacked layers. Without residual connections, it would be much harder to train such deep models effectively.

Stabilize Training: Combined with Layer Normalization, residual connections contribute to smoother learning and better convergence.

Layer Normalization

Layer Normalization normalizes the activations of a layer across the features of each individual input.
For each token (or position) in a sequence, it normalizes its vector of features:

For an input vector x of dimension d:

1. **Compute Mean (μ)**:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

2. **Compute Variance (σ2)**:

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

3. **Normalize:**

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

4. **Scale and Shift (learnable parameters):**

$$\text{LayerNorm}(x) = \gamma \cdot \hat{x} + \beta$$

In Transformers, Layer Normalization is usually applied after adding the residual connection. It works **independently per sample/token** (normalizes across features).

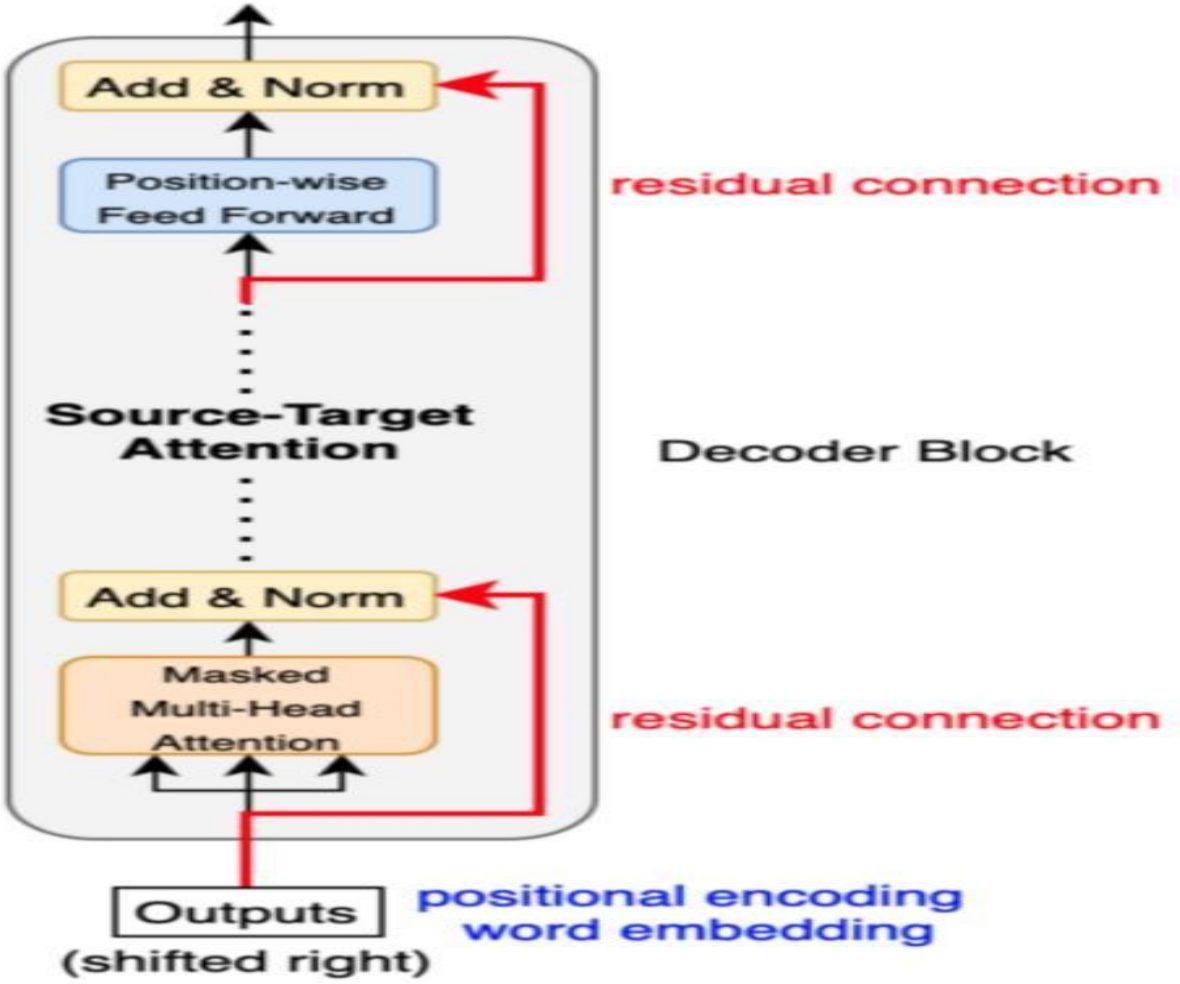
$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Layer Normalization prevents the mean and standard deviation of embedding vector elements from moving around, which makes training unstable and slow .

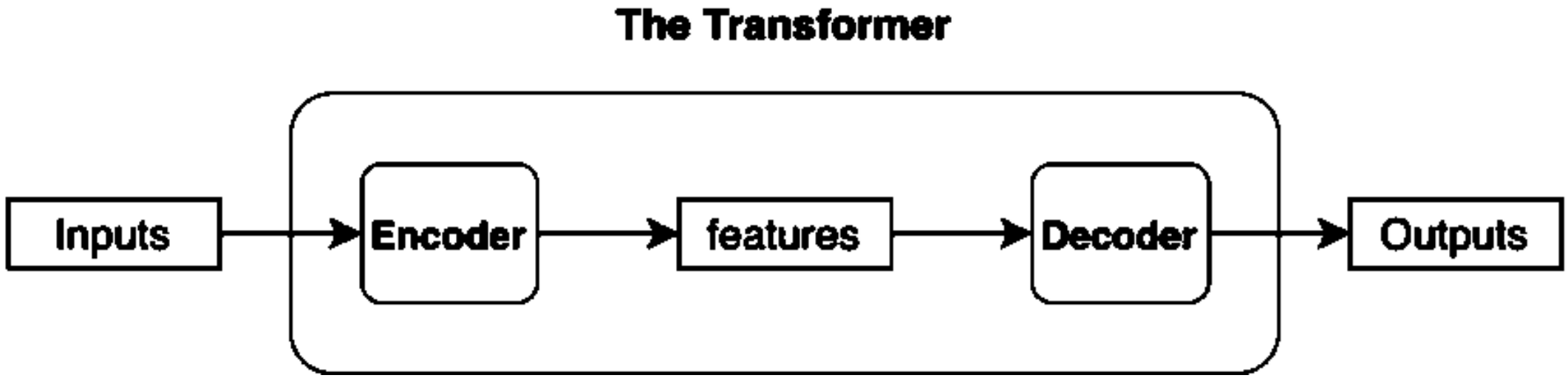
Decoder

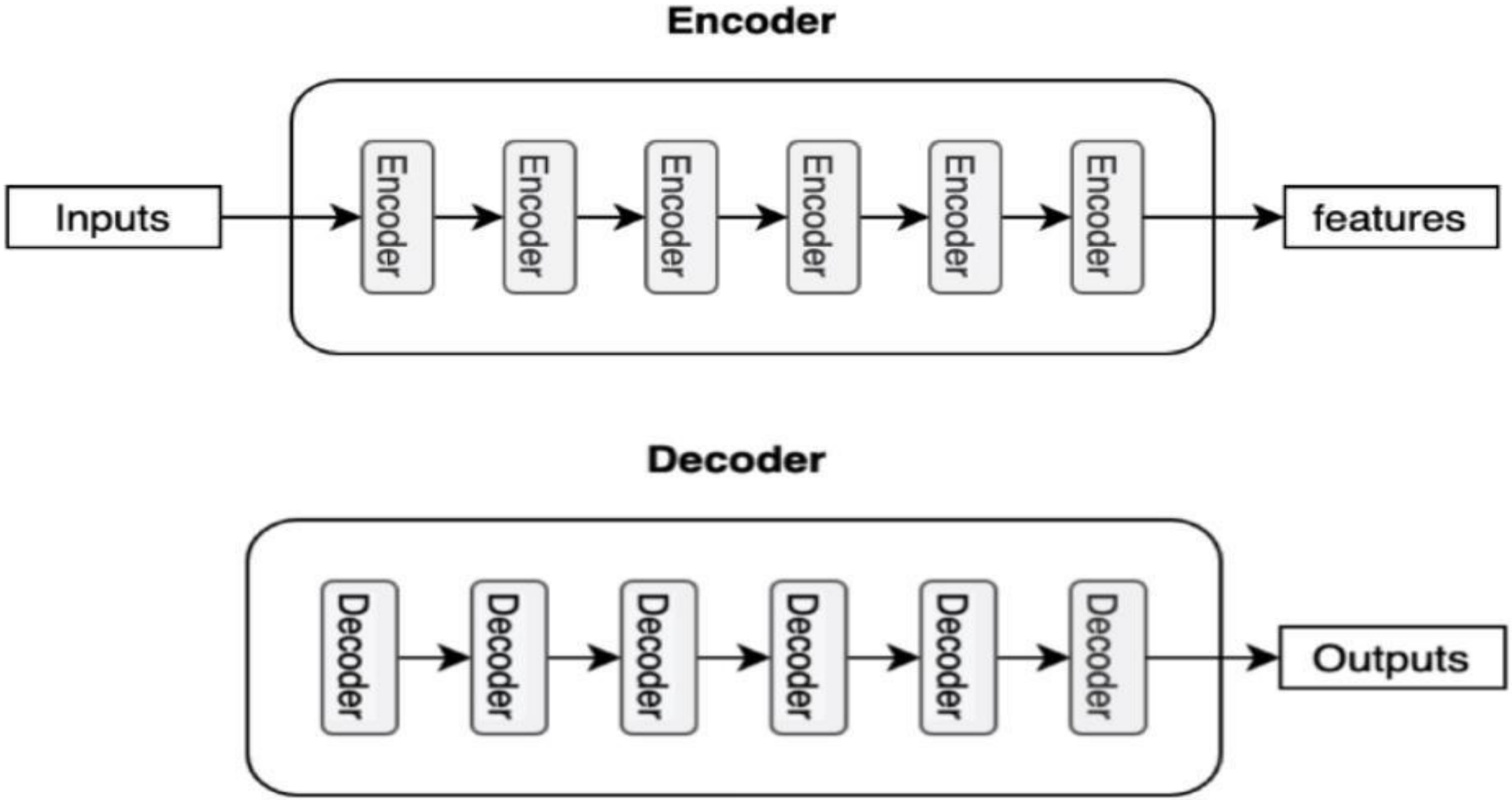
The decoder block is similar to the encoder block, except it calculates the source-target attention.

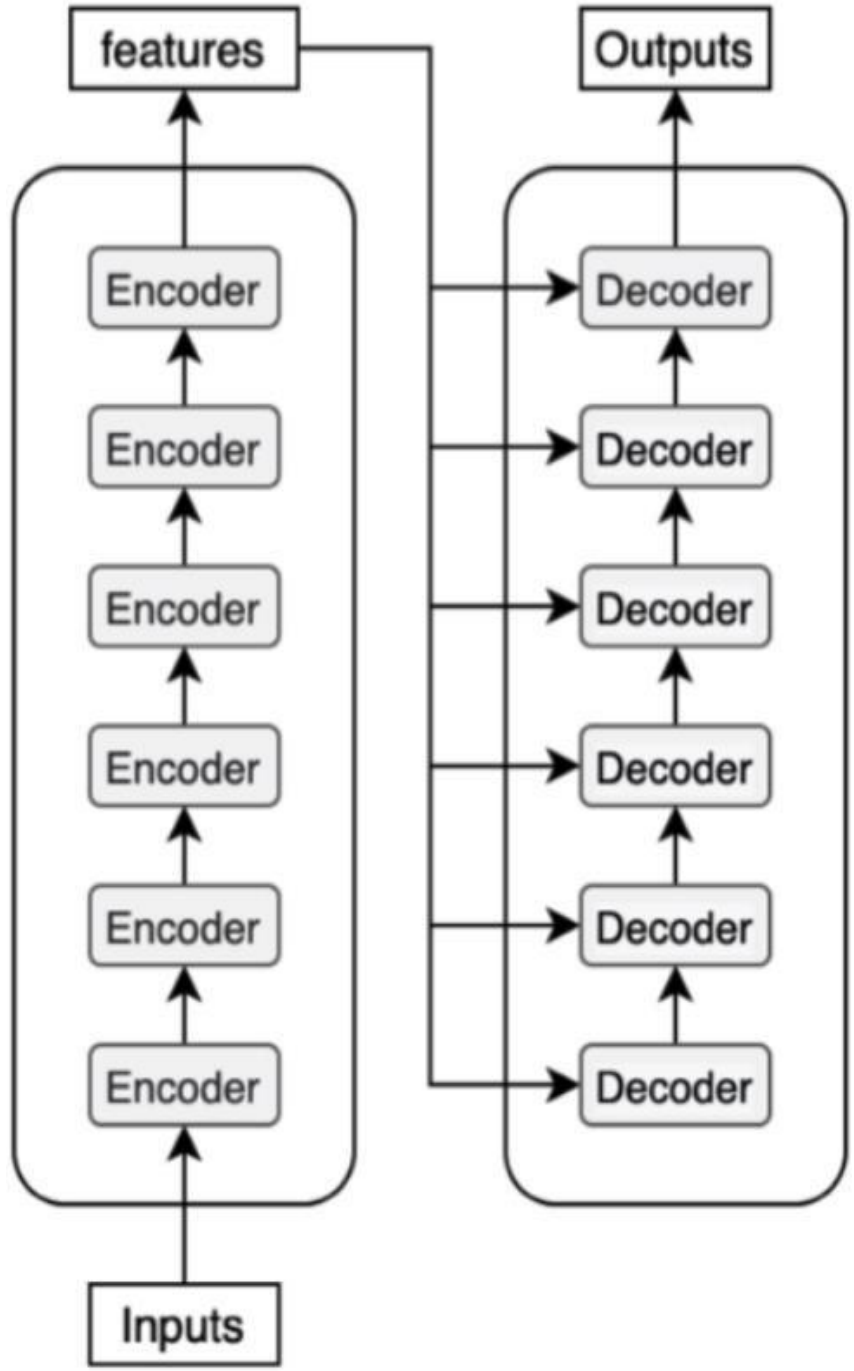
The source-target attention is another multi-head attention that calculates the attention values between the features (embeddings) from the input sentence and the features from the output (yet partial) sentence.

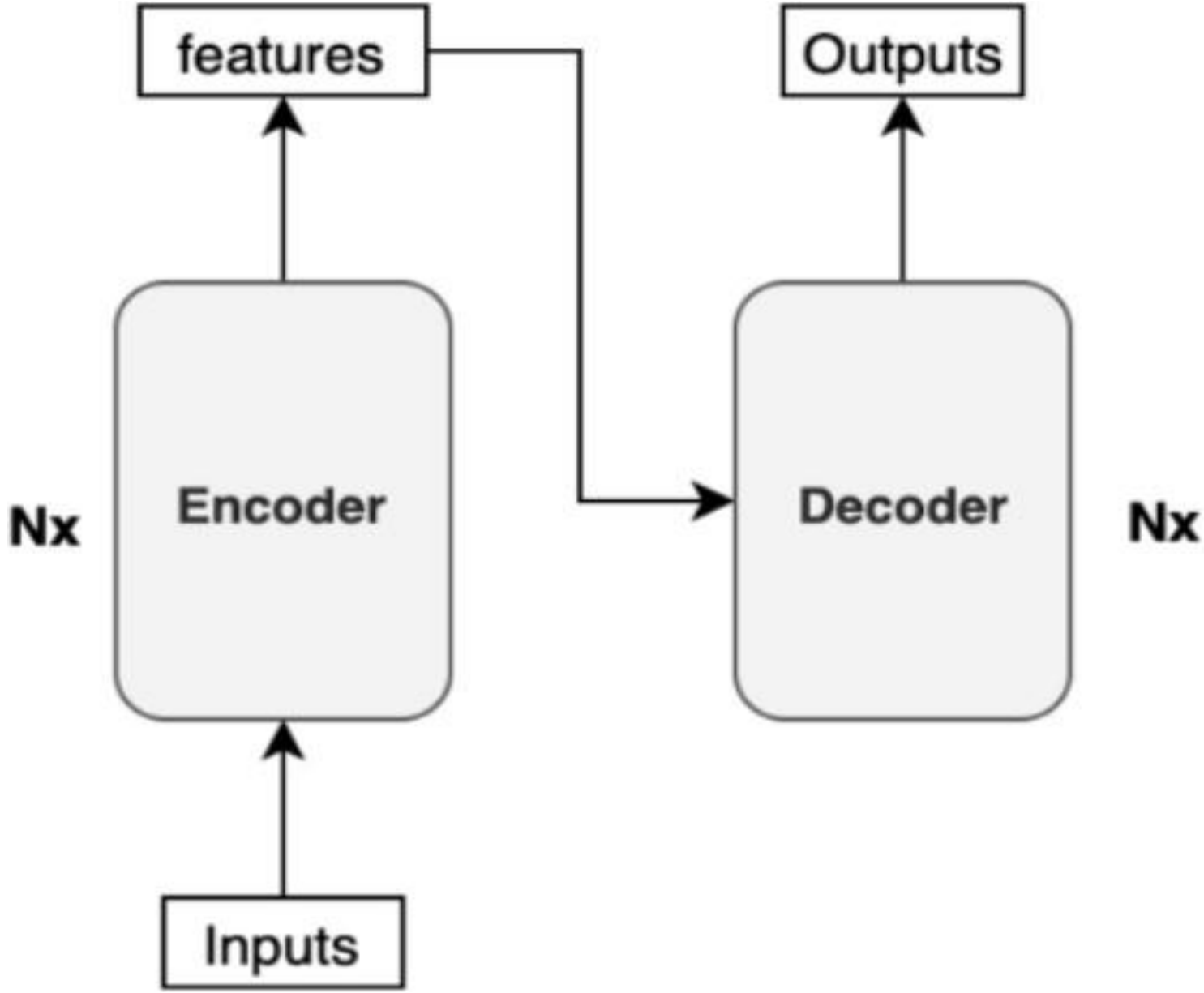


- The transformer architecture assumes no recurrence or convolution pattern when processing input data.
- As such, the transformer architecture is suitable for any sequence data.
- As long as we can express our input as sequence data, we can apply the same approach, including computer vision (sequences of image patches) and reinforcement learning (sequences of states, actions, and rewards).




Encoder
Inputs
Encoder
Encoder
Encoder
Encoder
Encoder
Encoder
features
Decoder
Decoder
Decoder
Decoder
Decoder
Decoder
Decoder
Outputs


features
Outputs
Encoder
Encoder
Encoder
Encoder
Encoder
Encoder
Decoder
Decoder
Decoder
Decoder
Decoder
Decoder
Inputs


features
Outputs
Nx
Encoder
Decoder
Nx
Inputs

This paper is all you need

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Loss Functions

- In a Transformer, the loss function measures how different the model's predicted output is from the correct target output.
- The loss is then used during backpropagation to update the model's weights.

1. **Language modeling:** Cross-Entropy Loss

For most Transformer language models such as GPT, the primary loss is cross-entropy loss.

Suppose the input is:" The cat is“

The model predicts probabilities for the next word:

Word	Predicted probability
sleeping	0.60
running	0.20
eating	0.15
jumping	0.05

If the actual next word is **"sleeping"**, the loss is:

$L = -\log(P(\text{correct word}))$

$L = -\log(0.60)$

$L \approx 0.511$

A higher probability for the correct word → lower loss.

2. General Formula

For a vocabulary of V words:

The loss function tells the Transformer how wrong its predictions are, and training tries to minimize this loss

$$L = -\sum_{i=1}^{V} y_i \log(\hat{y}_i)$$

where:

- y_i = actual/target probability
- $\hat{y}_i$ = predicted probability
- V = vocabulary size

For a sequence, the losses are calculated for each predicted token and usually averaged:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log P(y_t | y_1, \ldots, y_{t-1})$$

where T is the number of tokens.


Input Tokens
↓
Embedding + Positional Information
↓
Transformer Layers (Attention + FFN)
↓
Logits
↓
Softmax
↓
Predicted Probabilities
↓
Compare with Actual Target
↓
Cross-Entropy Loss
↓
Backpropagation
↓
Update Weights
↓
Repeat until Loss is minimized